



Capstone Project: Ft_Transcendence

Group Members' Names and Logins:

Fatima Al Zubaidi, faal-zub

Muna Almessabi, malmessa

Salma Alamoodi, saalamoo

Wadha Ahmed, wahmed

Date of Submission: 21 February 2025

1 Abstract

This work presents the design and development of Ft_Transcendence, a web application designed to create an engaging online gaming experience. The application integrates both traditional and modern web technologies to provide an intuitive interface, robust gameplay features, and efficient user management. The main challenge was developing a seamless platform that supports multiplayer interactions, includes a smart AI opponent, and features a dynamic tournament structure, while maintaining strong web integration, interactivity, and security. The project focuses on addressing key technical challenges related to security, scalability, and user experience. A primary goal was to implement secure user authentication through Two-Factor Authentication (2FA) and JSON Web Tokens (JWT), ensuring data protection across the platform. The development process followed a full-stack approach, utilizing Django for the back-end, PostgreSQL for data management, and Bootstrap for a responsive front-end design. Docker and Nginx were used to streamline containerization and deployment, ensuring the system's scalability and reliability. The project also applied best practices in software engineering, including risk assessments and structured testing methodologies, to ensure high performance and security. Key technical challenges included optimizing the user experience, balancing server load, and ensuring smooth gameplay for all users. This work contributes to the development of secure and scalable web applications, with potential applications in both gaming and user management systems. The project highlights the importance of combining strong security measures with user-centric design to create a reliable and engaging online experience.

2 Table of Contents

Contents

1	Abstract	1
2	Table of Contents	2
3	List of Figures	5
4	List of Abbreviations	6
5	Introduction	7
6	Software Development Life Cycle (SDLC)	8
6.1	Agile Adoption	8
6.2	Enhanced Communication	9
6.3	Task Organization	9
6.4	Code Quality	9
6.5	Role Flexibility	9
6.6	Final Integration	9
6.7	Deployment and Final Testing	10
6.7.1	Deployment Process	10
6.7.2	Final Testing	10
6.8	Gantt Chart	10
6.9	Risk Register and Risk Matrix	12
6.10	Functional Requirements	13
6.10.1	User Management	13
6.10.2	Gameplay	13
6.10.3	Tournament Feature	13
6.10.4	User Interface	13
6.10.5	Constraints	13
6.11	Technical Requirements	13
6.11.1	Hardware	13
6.11.2	Software	13
6.11.3	Security	14
6.11.4	Integration	14
6.11.5	Performance	14
6.11.6	Compliance aspects	14
6.12	Wireframes	14
6.13	Flowcharts	16
7	Requirement Analysis	18
7.1	Functional Requirements	18
7.2	Non-Functional Requirements	18
7.3	Constraints	19
7.4	Use Case Analysis	19
7.5	Justification of Module Choices	21
8	Design	22
8.1	System Architecture	22
8.2	User Interface (UI) Design	22
8.3	Security Design	22
8.4	Game Design	22
8.5	User Authentication and Authorization	23
8.6	Data Flow and Interaction Design	23
8.6.1	User Authentication and Authorization	23
8.6.2	Registration and Credential Verification	23
8.6.3	Gameplay Logic and Interaction	23

8.6.4	Game Results Storage	24
8.6.5	Overall	24
9	Implementation	25
9.1	Web Development	25
9.1.1	Backend Framework (Major Module)	25
9.1.2	Frontend Framework/Toolkit (Minor Module)	25
9.1.3	Database Integration (Minor Module)	25
9.2	User Management	25
9.2.1	Standard User Management (Major Module)	25
9.2.2	Remote Authentication (Major Module)	25
9.3	Gameplay and User Experience	25
9.3.1	Multiplayer Functionality (Major Module)	25
9.4	AI Algorithm	26
9.4.1	AI Opponent (Major Module)	26
9.4.2	User and Game Stats Dashboards (Minor Module)	26
9.5	Cybersecurity	26
9.5.1	Two-Factor Authentication (2FA) and JWT (Major Module)	26
9.6	DevOps	26
9.6.1	Microservices Architecture (Major Module)	26
9.7	Accessibility	26
9.7.1	Multiple Language Support (Minor Module)	26
9.7.2	Server-Side Rendering (SSR) Integration (Minor Module)	26
9.8	Development Environment	26
9.8.1	Tools and Technologies	27
9.8.2	Local Setup	27
9.8.3	Collaboration Tools	27
9.9	Integration and Testing	27
9.9.1	Integration	27
9.9.2	Testing	27
9.10	Deployment	27
9.10.1	Deployment Environment	27
9.10.2	Final Deployment	28
10	Testing	29
10.1	Testing Approach	29
10.2	Testing Phase	29
10.3	Documentation Procedures	29
11	Evolution	30
11.1	Our Project Evolution	30
11.2	Refinement Based on Testing and Optimization	30
11.3	Feature Development and Enhancements	30
11.4	Documentation and Future Plans	30
12	Conclusion	31
12.1	Areas for Improvement	31
12.2	Addressing Limitations	31
12.3	Final Thoughts	31
13	References	32
14	Appendices	33
14.1	1. Homepage	33
14.2	2. Sign Up	35
14.3	3. Sign In	35
14.4	4. Game Hub	36
14.5	5. Dashboard	37
14.6	6. Profile	38

14.7	7. Menu (Game Modes)	39
14.8	8. Settings	42

3 List of Figures

List of Figures

1	Gantt chart	11
2	Risk matrix	12
3	Risk register	12
4	Wireframe of a PingPong	15
5	Connection lines and navigation flow display the wireframe interconnections	16
6	Language Flowchart	17
7	PingPong Flowchart	17
8	Homepage	33
9	Instruction Page	33
10	Instruction Page - Video	34
11	About Us Page	34
12	Sign Up Page	35
13	Sign In Page	35
14	Verify Code (2FA) Page	36
15	Game Hub Page	36
16	Dashboard - User Stats and Match History	37
17	Friend Requests - Accept/Reject	37
18	User Profile	38
19	Game Menu - Select Game Mode	39
20	Single Player Setup	39
21	Two Player Setup	40
22	Multiplayer Setup	40
23	Tournament Setup	41
24	Settings Page	42
25	Enable 2FA	43
26	Disable 2FA	43
27	Edit Nickname	44
28	Edit Password	44
29	Delete Account	45
30	Language Settings	45

4 List of Abbreviations

Acronym	Definition
AI	Artificial Intelligence
2FA	Two-Factor Authentication
JWT	JSON Web Token
SQL	Structured Query Language
UI	User Interface
UX	User Experience
API	Application Programming Interface
HTML	HyperText Markup Language
Nginx	A high-performance web server and reverse proxy
Docker	A containerization platform for application deployment
Django	A high-level Python web framework for rapid development
PostgreSQL	A powerful open-source relational database management system
SDLC	Software Development Life Cycle
Gantt Chart	A type of bar chart that represents a project schedule
Agile	A software development methodology that emphasizes iterative development, collaboration, and flexibility
GitHub	A web-based platform for version control and collaboration using Git
Git	A distributed version control system for tracking changes in source code during software development
SSR	Server-Side Rendering
DevOps	Development and Operations
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
SSL	Secure Sockets Layer
ORM	Object-Relational Mapping
HTTPS	Hypertext Transfer Protocol Secure

5 Introduction

Ft_Transcendence is a comprehensive web application designed to deliver a robust and engaging multiplayer gaming experience. This project encompasses a wide range of modern web development practices, focusing on user management, gameplay, machine intelligence, cybersecurity, and Development and Operations (DevOps). Our goal was to create a scalable and secure platform that demonstrates proficiency in full-stack development.

The project leverages a sophisticated back-end framework, a dynamic front-end toolkit, and a relational database to provide a seamless user experience. Key features include standard and remote user authentication, multiplayer gameplay, an Artificial Intelligence (AI) opponent, and detailed user and game statistics dashboards. Security is paramount, with the implementation of Two-Factor Authentication (2FA) and JSON Web Tokens (JWT). Furthermore, we adopted a microservices architecture for the back-end, ensuring modularity and scalability.

This report provides a detailed overview of the development process, starting with the project's planning, as outlined in the Software Development Life Cycle (SDLC) section, which includes A type of bar chart that represents a project schedule (Gantt Chart) charts, risk assessments, and requirement specifications. Subsequent sections delve into the Requirement Analysis, Design, Implementation, Testing, and Evolution phases, highlighting the methodologies and technologies employed. The report begins with an abstract summarizing the project [1], followed by navigational aids such as the table of contents, lists of figures and tables, and abbreviations. It then proceeds to detail the Software Development Life Cycle (SDLC), requirement analysis, design choices, implementation processes, testing strategies, and evolutionary improvements, culminating in a comprehensive conclusion and supplementary appendices. This report aims to provide a comprehensive understanding of the Ft_Transcendence project, showcasing our technical skills and project management capabilities.

6 Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured framework used to guide the development of software systems, ensuring they are well-organized, efficient, and meet the desired technical and functional goals. In the context of the Ft_Transcendence project, the SDLC provides a systematic approach to software development, breaking the process into distinct phases, each with specific objectives, deliverables, and collaborative activities. By following the SDLC, our team was able to effectively manage the complexity of the project, mitigate potential risks, and deliver a robust and innovative solution that aligns with the learning objectives and technical challenges of the program. This approach not only enhanced our technical skills but also fostered teamwork, problem-solving, and a deeper understanding of real-world software development practices.

The SDLC typically consists of the following phases:

- **Planning:** In this phase, the project scope, objectives, and feasibility are defined. Key activities include resource allocation, timeline estimation, and risk assessment.
- **Analysis:** During this phase, the functional and technical requirements of the software are gathered and documented. This ensures a clear understanding of what the system should achieve.
- **Design:** The system architecture, user interface, and data flow are designed in this phase. Wireframes, flowcharts, and technical specifications are created to guide development.
- **Implementation (Development):** The actual coding and development of the software take place in this phase. Developers build the system based on the requirements and design documents.
- **Testing:** The software is rigorously tested to identify and fix defects. This ensures the system functions as intended and meets quality standards.
- **Deployment:** Once testing is complete, the software is deployed to the production environment and made available to end-users.
- **Maintenance:** After deployment, the system is monitored, and updates or fixes are applied as needed to ensure continued performance and relevance.

The SDLC is not a one-size-fits-all approach; it can be adapted to suit different project methodologies, such as Waterfall [2], Agile, or Iterative models. Each methodology emphasizes different aspects of the development process, but the core goal remains the same: to deliver high-quality software that meets user needs and business objectives. In the context of the Ping Pong Project, the SDLC provides a clear roadmap for developing a functional and user-friendly web-based game. By following the SDLC phases, the project team can ensure that all aspects of the game—from user functionalities and technical requirements to risk management and testing—are thoroughly addressed, resulting in a successful and enjoyable product for users.

6.1 Agile Adoption

Within our SDLC framework, we specifically chose the Agile methodology to manage our development process. This approach allowed us to implement the SDLC phases in an iterative manner, focusing on continuous delivery and adaptation. Our two-week sprint cycles incorporated elements from multiple SDLC phases simultaneously, with each sprint involving:

- Refinement of requirements (Analysis phase)
- Architectural decisions (Design phase)
- Feature implementation (Development phase)
- Comprehensive testing (Testing phase)
- Sprint demos (Mini-deployments)

This iterative approach allowed us to get early feedback on implemented features and adjust our direction as needed. For communication and coordination, we held bi-weekly online meetings using Zoom, complemented by daily asynchronous updates through our project management tool. This methodology proved particularly effective for our student team, allowing us to balance academic responsibilities while maintaining consistent progress on the project.

6.2 Enhanced Communication

As we approached critical milestones, we increased the frequency of our meetings to multiple times a week to stay on track and address any emerging issues. This shift allowed us to tackle challenges promptly, clarify misunderstandings, and adapt to changes in the project scope. Regular check-ins fostered a strong sense of accountability and teamwork, ensuring that we remained focused on delivering a high-quality product.

6.3 Task Organization

To manage our tasks effectively, we used a web-based platform for version control and collaboration using Git (GitHub) as our central project management tool [3]. Additionally, tools like Overleaf were used to streamline our workflow and maintain an organized record system [4]. Each team member created issues for their tasks, which we organized using labels like “To Do,” “In Progress,” and “Done.” This system provided a clear visual overview of the project’s status, making it easy to identify bottlenecks and allocate resources efficiently. GitHub’s Projects feature also helped us track progress across all modules, ensuring transparency and collaboration.

6.4 Code Quality

During the initial phase of the project, when we were meeting online, we established a distributed version control system for tracking changes in source code during software development (Git) workflow to ensure high code quality. Each developer worked on a separate branch, and before merging any code into the main branch, at least two team members reviewed and validated the changes. This process emphasized thorough testing and ensured that the code was stable and functional. However, when we transitioned to working on campus, our approach evolved. Instead of relying solely on remote reviews, we began checking the code together in person. Every edit or addition was reviewed collaboratively, and we tested the changes as a team to ensure they worked as intended. This hands-on approach not only improved the quality of the code but also strengthened our teamwork and problem-solving skills.

6.5 Role Flexibility

Initially, we divided the tasks based on our strengths: all members contributed to both front-end and back-end development for their respective modules. Two members managed the database for the back-end and worked on user management, including standard user management and users across tournaments. One member specifically handled remote authentication and the implementation of Two-Factor Authentication (2FA) and JSON Web Token (JWT) for cybersecurity and Server-Side Rendering (SSR). Another member focused on gameplay and user experience, implementing multiplayer functionality and working on the AI opponent. One member was responsible for the user and game stats dashboards and handled multiple language support for accessibility. Additionally, one member managed the game’s tournament and matchmaking system, allowing players to participate in tournaments, input alias names, and organize matches. All members collaborated in designing the back-end as microservices. However, as the project evolved, we frequently collaborated in multiple roles. For example, the member working on the tournament system assisted with debugging the gameplay logic, while the member focusing on gameplay supported the development of the matchmaking system. Similarly, the member working on the dashboards received support from the member handling the database to ensure the integration and accuracy of the data. This collaboration allowed us to tackle issues effectively and ensure all components were smoothly integrated.

6.6 Final Integration

In the final phase of the project, we transitioned to face-to-face meetings to integrate all modules, resolve dependencies, and refine the system. This hands-on collaboration was crucial for ensuring that the back-end, front-end, user management, and gameplay components worked together seamlessly. As we integrated each module, we continuously checked for bugs and tested the system thoroughly. Every time a bug was identified and fixed, we rechecked the entire system to ensure that the fix did not introduce new issues and that everything was functioning as expected. This meticulous approach allowed us to deliver a polished and fully functional Ping Pong game with advanced features like multiplayer support, AI opponents, tournament management, and robust security measures.

6.7 Deployment and Final Testing

After completing development, we deployed the application locally using Docker and Nginx to simulate a production environment. Docker allowed us to containerize the application, ensuring consistency across different environments, while Nginx served as a reverse proxy to handle requests and improve performance.

6.7.1 Deployment Process

1. We created Docker images for each component of the application.
2. Using Docker Compose, we orchestrated the containers to ensure seamless communication between services.
3. Nginx was configured to route traffic to the appropriate services and handle load balancing.

6.7.2 Final Testing

Once deployed, we conducted thorough testing to ensure all features worked as expected. This included:

- Testing individual modules to verify functionality.
- Performing end-to-end testing to ensure all components worked together seamlessly.
- Fixing any bugs or issues identified during testing to deliver a polished and fully functional application.

By the time of submission, we ensured the website was working smoothly, and all features were operational. This approach demonstrated our ability to deliver a complete and functional project, ready for evaluation.

6.8 Gantt Chart

The Gantt chart is a visual project management tool used to plan, schedule, and track tasks over time. It provides a clear overview of the project timeline, milestones, tasks, sub-tasks, dependencies, and ownership. In this section, we outline how the Gantt chart was utilized during the Planning phase to organize our project schedule and during the Monitoring and Control phase to track progress and manage dependencies effectively.

Figure 1: Gantt chart



6.9 Risk Register and Risk Matrix

The Risk Register and Risk Matrix are essential tools used during the Planning phase to identify potential risks, assess their impact, and develop mitigation strategies. For our Ping Pong project, we identified risks such as technical challenges, resource shortages, and timeline delays. Each risk was evaluated based on its likelihood and impact, and appropriate mitigation strategies were defined. During the Monitoring and Control phase, we regularly reassessed these risks, updated the Risk Register, and added new risks as the project evolved. This proactive approach helped us stay on track and address issues before they escalated.

Likelihood / Impact		Impact				
		Insignificant 1	Minor 2	Significant 3	Major 4	Severe 5
Likelihood	5 Almost Certain	Medium 5	High 10	Very high 15	Extreme 20	Extreme 25
	4 Likely	Medium 4	Medium 8	High 12	Very high 16	Extreme 20
	3 Moderate	Low 3	Medium 6	Medium 9	High 12	Very high 15
	2 Unlikely	Very low 2	Low 4	Medium 6	Medium 8	High 10
	1 Rare	Very low 1	Very low 2	Low 3	Medium 4	Medium 5

Figure 2: Risk matrix

Risk	Category	Mitigation Strategy	Contingency Plan	Responsibility	Likelihood (1-5) Low (1) – High (5)	Impact (1-5) Low (1) – High (5)	Severity Likelihood × Impact
Database migration failures leading to data loss	Technical	comprehensive backups before migration, test migration process, develop migration scripts	Prepare database rollback plan, maintain old database until successful migration confirmed	All	High (5)	High (5)	High (25)
User interface issues on browsers	User Experience	Cross-browser testing, responsive design implementation, use of standard Bootstrap components	Prepare simplified UI version, document known issues and workarounds	All	Medium (3)	Medium (3)	Medium (9)
Microservices communication failures	Infrastructure	Implement robust error handling, service discovery, and circuit breakers; monitor service health	Create fallback mechanisms, prepare degraded mode operation plan	All	Medium (3)	High (5)	High (15)
Authentication system failures affecting user access (e.g., login issues, password reset failures)	Security	Test authentication systems thoroughly, ensure fallback methods (e.g., password reset options) are available, use automated monitoring	Provide a backup authentication process, implement a helpdesk or support channel for manual resets, monitor systems for failures	All	High (5)	Medium (3)	High (15)
Limited storage on company computers affecting system performance or deployment	Infrastructure	Use higher storage computers when available, regularly clear goinfre, move working files to goinfre or perform periodic backups to personal laptops	If storage limits are exceeded, delete non-essential files, archive completed tasks to external drives, and maintain routine storage checks	All	Low (2)	High (4)	Medium (8)

Figure 3: Risk register

6.10 Functional Requirements

Functional requirements define what the system should do and how users will interact with it. They include tasks, user functionalities, interactions, constraints, and use cases.

6.10.1 User Management

- Users can register and log in.
- Users can authenticate remotely (42 Application Programming Interface (API) intra account).
- Users can update their profiles and view match history.

6.10.2 Gameplay

- Users can play the basic 2 player pong game.
- Users can play against an AI opponent.
- Users can play multiplayer games with 1-6 players. With 1 player having to survive the speeding ball fall a complete minute.
- Users can view real-time scores and game statistics.

6.10.3 Tournament Feature

- Users can create and join tournaments.
- Tournament results are continuously updated and displayed.

6.10.4 User Interface

- The interface supports multiple languages.
- The layout is intuitive and easy to navigate.

6.10.5 Constraints

- The system must support a lot of concurrent users.
- The game must respond to user inputs with minimal latency.

6.11 Technical Requirements

Technical requirements define how the system will achieve its functionality. They include hardware, software, security, integration, performance, and compliance aspects.

6.11.1 Hardware

- Servers with sufficient capacity to handle real-time multiplayer gameplay.
- Backup servers for redundancy.

6.11.2 Software

- Backend: Use a framework (Django) and design microservices.
- Frontend: Use a front-end framework or toolkit (Bootstrap).
- Database: Use a relational database (PostgreSQL).

6.11.3 Security

- Implement two-factor authentication (2FA) for user accounts.
- Use JSON Web Tokens (JWT) for secure session management.
- Encrypt sensitive data (passwords) using standard protocols.

6.11.4 Integration

- Integrate server-side rendering (SSR) for improved performance.
- Integrate third-party authentication services (42 intra).

6.11.5 Performance

- Ensure the game responds to user inputs with less than 100ms latency.
- Optimize server performance to handle a lot of concurrent users.

6.11.6 Compliance aspects

- Ensure compliance with GDPR for user data protection by implementing secure data storage in PostgreSQL and proper user consent mechanisms for data collection. Copy
- Implement accessibility features in the user interface to support multiple languages and provide an inclusive experience for all users.
- Adhere to security best practices through the implementation of Two-Factor Authentication (2FA) and JWT token-based authentication to protect user accounts and sessions.
- Follow Django's security guidelines for protection against common web vulnerabilities such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).
- Comply with the terms of service for third-party integrations, specifically the 42 API intra account authentication.
- Ensure proper licensing for all open-source components used in the project, including Django, Bootstrap, and other libraries.

6.12 Wireframes

Wireframes are visual blueprints of the application's user interface, providing a clear representation of each screen's layout and functionality [5]. They serve as a foundational design tool, illustrating the placement of key elements such as buttons, menus, and content areas, as well as the sequential user navigation flow. In the context of our Ping Pong project, wireframes were created to ensure a clean, intuitive, and visually consistent User Interface (UI). By mapping out critical screens—such as the homepage, login/registration page, player profile, tournament lobby, and game screen—we were able to experiment with different design solutions and refine the placement of interactive elements. The wireframes emphasized visual hierarchy, spacing, and consistency across screens, ensuring that the interface was both functional and aesthetically pleasing. Annotated with descriptions, the wireframes helped the team visualize the structure of the game and provided a clear guide for development. This process ensured that the final product delivered a polished and user-friendly interface, aligning with the principles of effective UI design.

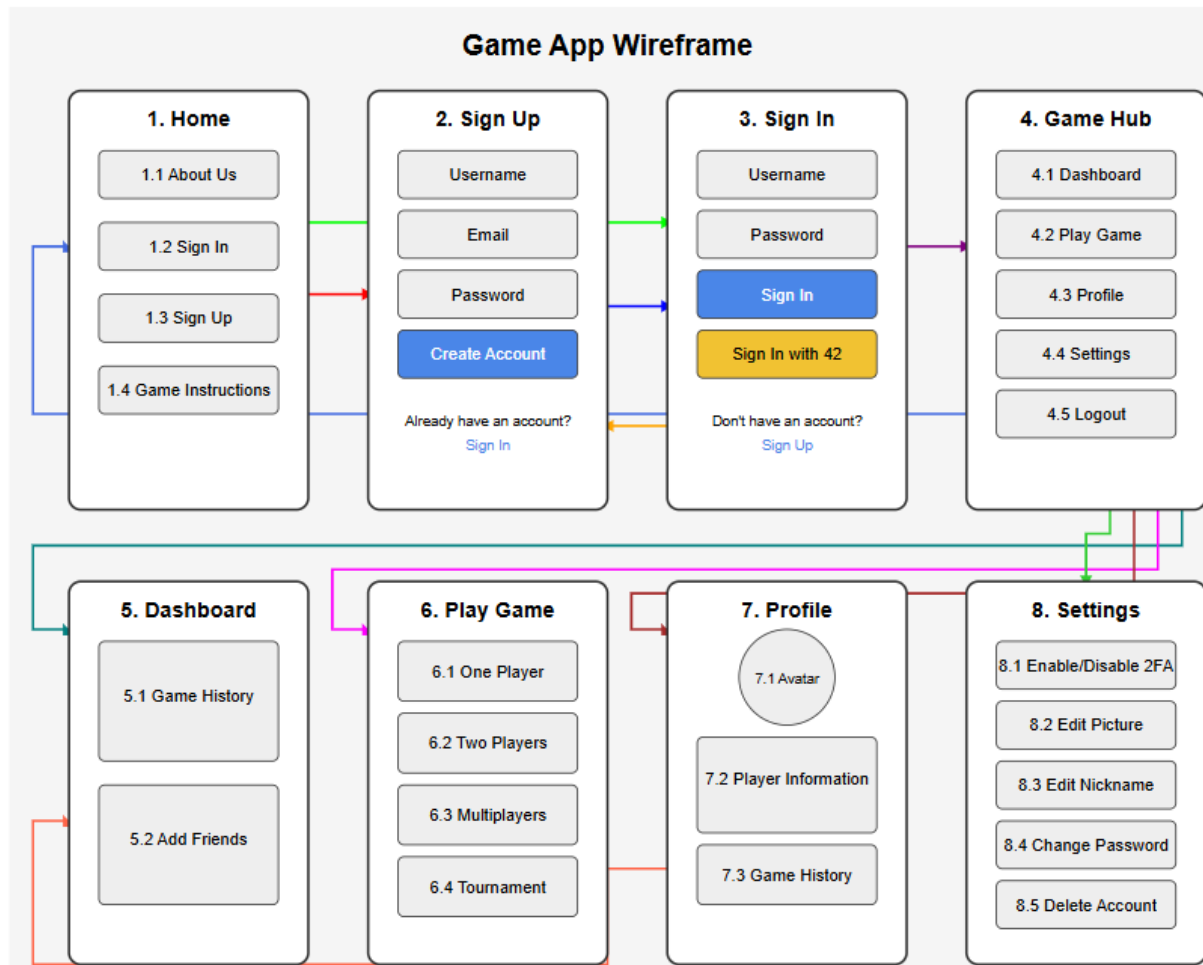


Figure 4: Wireframe of a PingPong

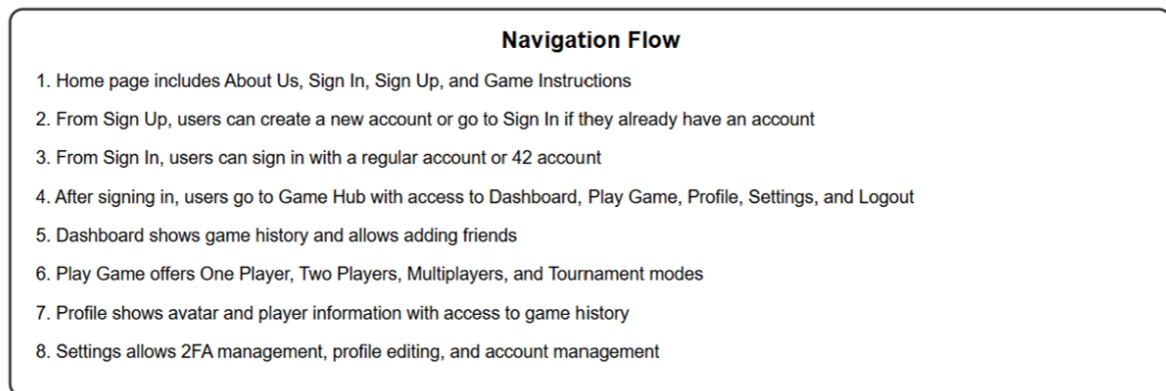
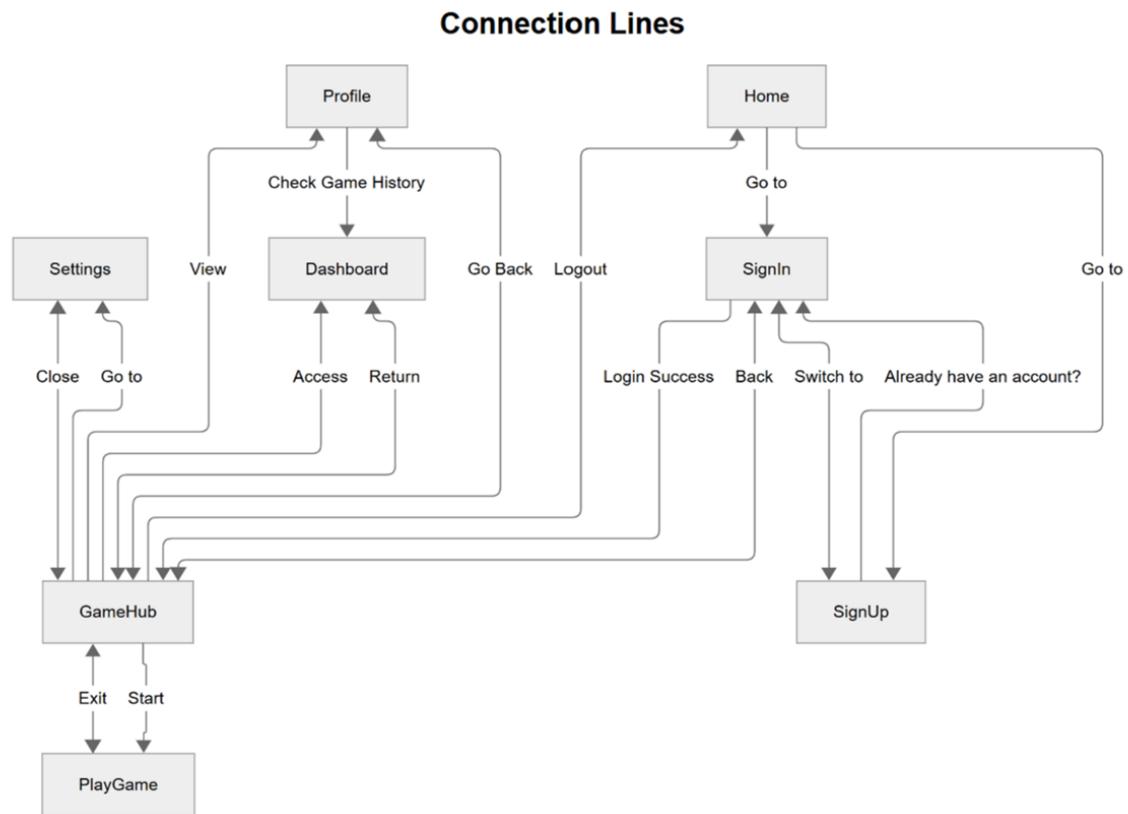


Figure 5: Connection lines and navigation flow display the wireframe interconnections

6.13 Flowcharts

Flowcharts are visual diagrams that illustrate the primary user interactions, system processes, and data flow within the application [6]. They provide a comprehensive overview of the project's functional workflow, mapping out how users navigate through key components such as the homepage, settings, player profile, and game sections. Flowcharts also highlight the logical flow of data between different parts of the system, ensuring that all processes are clearly defined and aligned with the project's requirements. For our Ping Pong project, flowcharts were created to visualize the user journey, from logging in and joining a game to interacting with the AI opponent or multiplayer features. These diagrams served as a critical tool for understanding and refining the system's functionality before implementation.

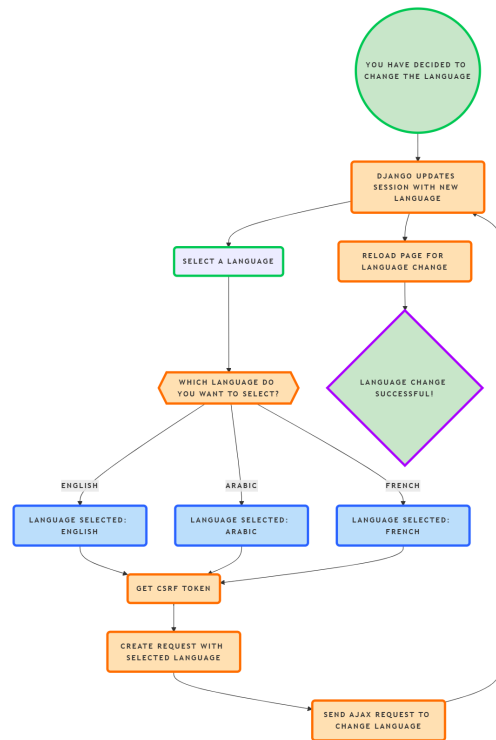


Figure 6: Language Flowchart

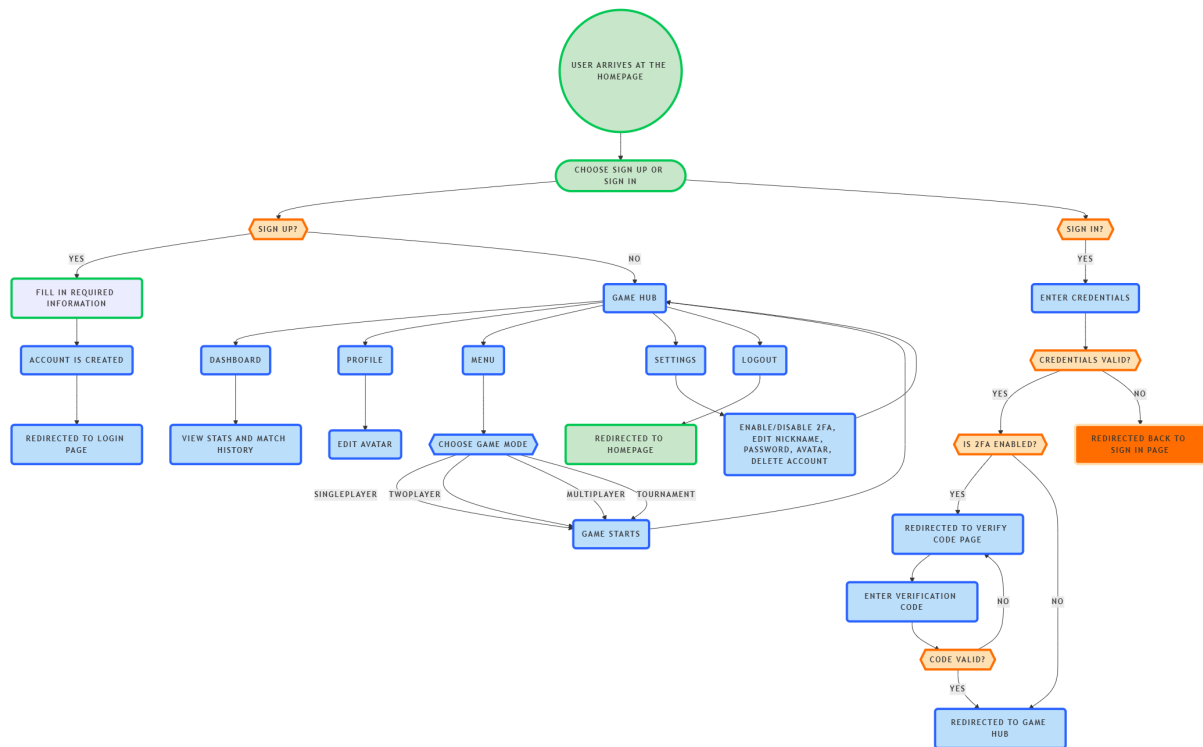


Figure 7: PingPong Flowchart

7 Requirement Analysis

This document details the requirements for the Ft_transcendence website, a platform enabling users to participate in Pong games. It outlines functional and non-functional requirements, project constraints, and a use case analysis. This analysis serves as the foundation for the design and development phases of the project.

7.1 Functional Requirements

- **User Management:**
 - Users can create accounts, log in, and manage profiles (display name, email, avatar).
 - Users can update profile information, including enabling/disabling 2FA, changing passwords or nicknames, and deleting their accounts.
 - Users can add friends, view their dashboards, and manage friend lists.
 - User dashboards display statistics (wins/losses, scores, games played) and match history, potentially including performance history visualized as a chart or diagram.
- **Tournament System:**
 - Users can register for tournaments.
 - The system manages tournament brackets/matchmaking and notifies users of upcoming matches.
 - Tournament progress and scores are tracked and displayed.
- **Gameplay:**
 - Users can play Pong against other users in real-time.
 - (Chosen Module) Users can play against an AI opponent.
 - (Chosen Module) Users can participate in multiplayer Pong games (1-6 players).
- **Statistics and Data:**
 - (Chosen Module) User dashboards display detailed game statistics.
 - (Chosen Module) Game dashboards display match statistics (date played, game type, outcome, scores).
- **Security:**
 - (Chosen Module) Two-factor authentication (2FA) and JWT for user login and secure communication.

7.2 Non-Functional Requirements

- **Performance:**
 - The website loads quickly and provides a responsive user experience.
 - (Chosen Module) Server-side rendering (SSR) to improve initial load times.
- **Security:**
 - User passwords must be securely hashed (PostgreSQL).
 - Protection against common web vulnerabilities, including Cross-Site Request Forgery (Cross-Site Request Forgery (CSRF), Cross-Site Scripting (XSS)).
 - (Chosen Module) Two-factor authentication for enhanced security.
- **Usability:**
 - The website is intuitive and easy to navigate.
 - (Chosen Module) Multi-language support.
- **Scalability:**
 - The platform should be able to handle a growing number of users and tournaments. (The microservices architecture chosen in the DevOps module addresses this requirement.)
- **Maintainability:**
 - Code be well-structured and documented for easy maintenance and updates.

7.3 Constraints

- **Technology:**
 - **Backend:** Django (robust features, scalability, and suitability for web application development) [7].
 - **Frontend:** Bootstrap (responsive design capabilities, ease of use, and ability to accelerate frontend development).
 - **Database:** PostgreSQL (reliability, performance, and support for advanced features).
- **Deployment:** The project be deployable using Docker.
- **Security:** Adherence to security best practices (password hashing, input validation, 2FA, JWT).

7.4 Use Case Analysis

- **Register User (User Management Module):**
 - **Actor:** New User
 - **Precondition:** User has not created an account.
 - **Main Success Scenario:**
 - User navigates to the registration page.
 - User provides required information (username, firstname, lastname email, password).
 - System validates input.
 - System creates a new user account.
 - User is logged in.
 - **Alternative Flows:**
 - Invalid input (username characters, incorrect email format, confirm Passwords do not match).
 - Username already exists.
 - Email already exists.
- **Login User (User Management Module, Cybersecurity Module):**
 - **Actor:** Registered User, Admin
 - **Precondition:** User has an account.
 - **Main Success Scenario:**
 - User navigates to the login page.
 - User provides credentials (username and password).
 - System validates credentials.
 - System generates a session token (JWT).
 - User is logged in.
 - **Alternative Flows:**
 - Incorrect credentials.
 - 2FA (if enabled).
 - Account not found.
- **Play Game (vs. AI) (Gameplay and User Experience Module, AI-Algo Module):**
 - **Actor:** Registered User, Admin
 - **Precondition:** User is logged in.
 - **Main Success Scenario:**
 - User navigates to the game selection page.
 - User selects "One Player"
 - System initiates the game.
 - User plays against the AI opponent.

- Game ends.
- System records the game results (win/loss, score).
- **Play Game (Multiplayer) (Gameplay and User Experience Module):**
 - **Actor:** Registered User, Admin
 - **Precondition:** User is logged in.
 - **Main Success Scenario:**
 - User navigates to the multiplayer game settings.
 - Write the name for players to join the game.
 - Game starts.
 - Players play the game.
 - Game ends.
 - System records the game results(win/loss).
- **View Statistics (User and Game Stats Dashboards Module):**
 - **Actor:** Registered User, Admin
 - **Precondition:** User is logged in.
 - **Main Success Scenario:**
 - User navigates to their profile/dashboard.
 - System displays statistics (wins/losses, scores, games played), match history, and potentially a performance chart.
- **Manage Friends (AI-Algo Module):**
 - **Actor:** Registered User, Admin
 - **Precondition:** User is logged in.
 - **Main Success Scenario:**
 - User navigates to the "Friends" section.
 - User searches for another user.
 - User sends a friend request.
 - Other user accepts/declines the request.
 - **Alternative Flows:**
 - User not found.
 - Request already sent.
 - You are already friends.
 - You cannot send a friend request to yourself.
- **2FA Management (Cybersecurity Module):**
 - **Actor:** Registered User, Admin
 - **Precondition:** User has an account.
 - **Main Success Scenario (Enabling 2FA):**
 - User navigates to their settings.
 - User selects the "Enable 2FA" option.
 - User enters their current password for verification.
 - **Main Success Scenario (Login with 2FA):**
 - User navigates to the login page.
 - User enters their username and password.
 - System verifies the credentials.
 - System prompts for the 2FA verification code.
 - System sends a time-based code to the user's registered email address.

- User enters the code received via email.
- System verifies the code.
- User is logged in.
- **Main Success Scenario (Disabling 2FA):**
 - User navigates to their settings.
 - User selects the "Disable 2FA" option.
 - User enters their current password for verification.
 - System disables 2FA for the user's account.
- **Alternative Flows:**
 - Incorrect password entered during 2FA enabling/disabling.
 - Invalid verification code during 2FA enabling/login.
 - Code expired or incorrect during login.

7.5 Justification of Module Choices

This section will detail the rationale behind each chosen module, including the framework, frontend toolkit, and database.

- **Django Framework:** Chosen for its Model-View-Template (MVT) architecture, which promotes code organization and maintainability. Its built-in features for user authentication, database management significantly reduce development time and effort.
- **Bootstrap Toolkit:** Selected for its responsive grid system and pre-built components, which enable rapid development of a visually appealing and mobile-friendly user interface.
- **PostgreSQL Database:** Chosen for its reliability, data integrity features, and scalability, making it suitable for handling a large number of users and game data.
- **Microservices Architecture:** implemented to enhance scalability and maintainability. The backend is divided into three independent services, each containerized with Docker. App 1 handles core website functions (homepage, about us, instructions, authentication). App 2 manages user-specific data (dashboards, game hub, profiles). App 3 focuses on gameplay and navigation (menu, multiplayer, single-player, tournaments). This design allows independent scaling and updates, improving resilience and future development. This approach facilitates more efficient resource allocation and reduces the impact of failures in one service on the others.
- **JWT (JSON Web Tokens):** Implemented for secure authentication and authorization. JWT provides a stateless and efficient way to verify the identity of users and grant access to protected resources.

8 Design

This section provides a comprehensive overview of the architectural and design decisions behind the web application, emphasizing the creation of a robust, scalable, and secure platform. We will explore key areas of the system's architecture, component design, user interface (UI) design, security strategies, and gameplay mechanics. Additionally, we will discuss the design patterns, best practices, and performance optimization techniques that contribute to the platform's overall functionality and user experience.

8.1 System Architecture

To achieve modularity and scalability, we implemented a microservices architecture for the backend. This subsection details the overall system architecture, including service interaction patterns and communication technologies. In addition, and to facilitate organized development and maintenance within our microservices framework, we adopted the Django application structure. This approach enabled us to logically separate concerns, promote code reusability, and simplify the deployment and scaling of individual services.

Complementing Django's structure, Dockerization was employed to effectively realize the microservices architecture. Each service was containerized, ensuring isolation and independent deployment, thereby enabling seamless scaling and management. On the other hand, for the front-end, we utilized the Bootstrap framework to expedite UI development and guarantee a responsive design. Bootstrap's pre-built components and grid system significantly accelerated our development process, allowing us to create a consistent and user-friendly interface.

8.2 User Interface (UI) Design

Our primary goal was to create a user-friendly and engaging interface for Ft_Transcendence. To maintain consistency across the application, we used a base HTML file as a foundation for the UI design. However, we made intentional modifications throughout the development process to ensure that the user experience was prioritized, rather than sticking rigidly to a single, fixed design.

We carefully designed buttons to facilitate easy navigation between pages, ensuring that users can quickly and clearly access any page they intend to visit. The connections between pages were also optimized for smooth transitions, minimizing confusion and enhancing the overall flow of the website. In terms of visual design, we thoughtfully selected colors, fonts, and images to not only align with the brand but also to boost the user experience and increase overall enjoyment. Every element was chosen with the intent to create an aesthetically pleasing and intuitive interface, ensuring the application is as enjoyable to use as it is functional.

8.3 Security Design

Security is a critical aspect of our system, and we made it a top priority throughout the development process. To safeguard user data and protect against unauthorized access, we implemented several key security measures. One of the primary features we introduced is Two-Factor Authentication (2FA), which adds an extra layer of protection by requiring users to verify their identity through both their password and a secondary authentication method. This significantly reduces the risk of unauthorized access.

Additionally, we employed JSON Web Tokens (JWT) for secure user authentication and session management. JWT provides a robust and scalable way to handle user login sessions, ensuring that sensitive data is transmitted securely between the client and the server. We also incorporated other security best practices, such as encrypting sensitive data and using secure communication protocols like Hypertext Transfer Protocol Secure (HTTPS) to ensure that all data exchanged between users and the server remains private and secure.

8.4 Game Design

This section delves into the design of the AI opponent and the various gameplay features implemented to offer a challenging and immersive experience. We dedicated significant time to enhancing the gameplay options, ensuring that each mode was engaging and well-balanced. The game includes several modes, each with unique mechanics and challenges:

- **Basic Pong Game:** A simple and classic version of the traditional Pong game, offering a nostalgic experience while maintaining a clean and user-friendly interface.

- **AI Opponent:** An AI-controlled opponent that mimics human behavior, adjusting its strategies to offer a progressively more challenging experience as the game advances, ensuring a competitive yet fun environment for players.
- **Single-Player Mode:** Designed for solo players, this mode challenges the player by dynamically adjusting the speed and time of the game. The player's movement range is also expanded to improve the overall gameplay experience, making it more enjoyable and engaging.
- **Multiplayer Mode:** This mode supports between 1 to 6 players, with the gameplay adapting based on the chosen number of participants. It provides flexibility, ensuring that each game session is well-balanced and appropriately tailored to the number of players involved.

Throughout all modes, we maintained a consistent theme, with subtle modifications in the multiplayer mode to allow for flexibility depending on the number of players. The overall design integrates speed adjustments, color schemes, movement tracking, control keys, and other gameplay mechanics in a logical and user-friendly way, improving both the playability and the overall user experience.

8.5 User Authentication and Authorization

User authentication is critical for security. The registration process begins when a user submits their credentials via the front-end form. The front-end sends a request to the authentication microservice, which verifies the credentials against the database. Upon successful registration, a JSON Web Token (JWT) is generated and returned to the client, enabling secure access to resources. Two-Factor Authentication (2FA) is also implemented for added security.

8.6 Data Flow and Interaction Design

This section outlines the flow and interaction of data within the application, focusing on user authentication, gameplay logic, and data storage.

8.6.1 User Authentication and Authorization

User authentication is critical for security. The registration process begins when a user submits their credentials via the front-end form. The front-end sends a request to the authentication microservice, which verifies the credentials against the database. Upon successful registration, a JSON Web Token (JWT) is generated and returned to the client, enabling secure access to resources. Two-Factor Authentication (2FA) is also implemented for added security.

8.6.2 Registration and Credential Verification

The user enters their details (e.g., username, password, etc.) on the registration form. The front-end sends this data to the authentication microservice via an HTTP POST request. The authentication service checks the provided credentials against the database. If they are valid, the process continues. In case of invalid credentials, an error message is sent back to the user.

Upon successful verification, the authentication microservice generates a JWT containing user information. If 2FA is enabled, the user must complete an additional verification step before the JWT is issued. After receiving the JWT, the client includes it in the "Authorization" header for subsequent requests. The backend services validate the JWT to ensure the user has the necessary permissions to access the requested resources.

8.6.3 Gameplay Logic and Interaction

The gameplay logic is responsible for managing the interactions between players and the game environment. When a player starts a game, the front-end sends the game initialization request to the backend. The backend then sets up the game state, including the player's position, game speed, and other relevant parameters. During the game, the front-end sends regular updates on player actions (such as movements and key presses) to the backend, which processes these actions and updates the game state accordingly. The backend then sends the updated game state back to the front-end to render the new positions and actions.

8.6.4 Game Results Storage

At the end of each game session, the results, including the player's score, time played, and any other relevant data, are sent from the front-end to the backend for storage. The backend stores these results in a database to maintain player progress and track historical performance. The game results are associated with the user's account, allowing them to view their past achievements and scores. The results may also be used for ranking purposes in multiplayer games or tournaments.

8.6.5 Overall

The integration of user authentication, gameplay logic, and game results storage has been carefully designed to ensure a seamless and secure user experience. Each component works cohesively within the application, from verifying user credentials and maintaining secure access through JWT and 2FA, to managing real-time gameplay interactions and storing game results for future reference. This thoughtful and systematic integration ensures that the Ft_Transcendence application not only provides an engaging gaming experience but also guarantees the reliability and security necessary for modern web applications.

9 Implementation

9.1 Web Development

This subsection covers the implementation of the web-based components of the project, including the backend, frontend, and database.

9.1.1 Backend Framework (Major Module)

- **Implementation:** We used Backend Framework: Django to build the backend of the application. The framework was chosen for its scalability and support for real-time communication, which was essential for multiplayer functionality.
- **Details:** The backend handles apps management as well as the communication between the frontend and database.

9.1.2 Frontend Framework/Toolkit (Minor Module)

- **Implementation:** The frontend was developed using Bootstrap, which provided a responsive and dynamic user interface.
- **Details:** Bootstrap's grid system and pre-designed components were utilized to ensure a mobile-first and accessible design.

9.1.3 Database Integration (Minor Module)

- **Implementation:** We used Database: PostgreSQL to store user data, game statistics, and tournament information.
- **Details:** The database was designed to support efficient querying and scalability, with tables/collections for users, matches, and tournaments.

9.2 User Management

This subsection focuses on user authentication, profile management, and remote authentication.

9.2.1 Standard User Management (Major Module)

- **Implementation:** Users can register, log in, and manage their profiles. This includes viewing match history and scores.
- **Details:** User data is stored securely in the database, and authentication is handled using JWT (JSON Web Tokens).

9.2.2 Remote Authentication (Major Module)

- **Implementation:** We integrated remote authentication using API 42 intra to allow users to log in with third-party accounts.
- **Details:** This feature enhances user convenience and reduces the need for manual account creation.

9.3 Gameplay and User Experience

This subsection covers the implementation of multiplayer functionality and user experience enhancements.

9.3.1 Multiplayer Functionality (Major Module)

- **Implementation:** The game supports multiplayer mode with 1-6 players in the same game. Game functionality is handled using math logic.
- **Details:** Players can join games, interact in real-time, and view live scores.

9.4 AI Algorithm

This subsection describes the implementation of the AI opponent and game statistics dashboards.

9.4.1 AI Opponent (Major Module)

- **Implementation:** An AI opponent was introduced using raytracing logic. The AI adapts to the player's skill level for a challenging experience.
- **Details:** The AI logic is integrated into the backend and communicates with the frontend to display moves in real-time.

9.4.2 User and Game Stats Dashboards (Minor Module)

- **Implementation:** Dashboards were created to display user statistics (win/loss ratio) and game data (match history).
- **Details:** Data is fetched from the database and displayed using charts and tables on the frontend.

9.5 Cybersecurity

This subsection highlights the implementation of security measures.

9.5.1 Two-Factor Authentication (2FA) and JWT (Major Module)

- **Implementation:** 2FA was implemented using Email, and JWT was used for secure session management.
- **Details:** These measures ensure that user accounts are protected from unauthorized access.

9.6 DevOps

This subsection covers the backend architecture and deployment.

9.6.1 Microservices Architecture (Major Module)

- **Implementation:** The backend was designed as microservices to improve scalability and maintainability. Each service (e.g., user management, game logic) operates independently.
- **Details:** Microservices are deployed using containerization (Docker).

9.7 Accessibility

This subsection focuses on accessibility features.

9.7.1 Multiple Language Support (Minor Module)

- **Implementation:** The application supports multiple languages using i18n library.
- **Details:** Users can switch languages via a dropdown menu in the settings.

9.7.2 Server-Side Rendering (SSR) Integration (Minor Module)

- **Implementation:** SSR was integrated using Django views to improve performance.
- **Details:** SSR ensures faster page loads and better search engine visibility.

9.8 Development Environment

This subsection describes the tools, technologies, and setup used during development.

9.8.1 Tools and Technologies

- **Backend:** [Framework: Django]
- **Frontend:** [Framework: Bootstrap]
- **Database:** [Database: PostgreSQL]
- **Version Control:** GitHub for code management and collaboration.
- **IDE:** [VS Code]

9.8.2 Local Setup

- Developers set up local environments with all necessary dependencies installed.
- Docker was used to containerize the application for consistent development environments.

9.8.3 Collaboration Tools

- GitHub Projects for task management and issue tracking.
- Regular team meetings (online and in-person) to discuss progress and resolve issues.

9.9 Integration and Testing

This subsection covers how different modules were integrated and tested to ensure functionality and reliability.

9.9.1 Integration

- **Frontend-Backend Integration:** Real-time features (e.g., multiplayer) were implemented using Django Views.
- **Database Integration:** The database was connected to the backend using Django's Object-Relational Mapping (ORM).
- **Third-Party Services:** Remote authentication (42 intra) and 2FA services were integrated into the user management module.

9.9.2 Testing

- **Unit Testing:** Individual components (e.g., AI algorithm, authentication) were manually tested by enabling Django's DEBUG=True mode, which allowed for error tracking and debugging during development.
- **Integration Testing:** End-to-end testing was performed to ensure all modules worked together seamlessly.
- **Performance Testing:** Load testing was conducted to ensure the system could handle at least 20 concurrent users.
- **Bug Fixing:** Identified bugs were logged, prioritized, and fixed during regular testing cycles.

9.10 Deployment

This subsection explains how the application was deployed and made available to users.

9.10.1 Deployment Environment

- **Backend:** Deployed using Django in a Docker container, with Nginx as the reverse proxy server for managing traffic. PostgreSQL is used as the database for data storage.
- **Frontend:** Developed using Bootstrap for responsive design and hosted on the appropriate server, ensuring fast and reliable delivery.
- **Database:** Managed using PostgreSQL, connected to the backend through Docker containers for a streamlined environment.

9.10.2 Final Deployment

- The application was deployed to the production environment after thorough testing.
- Domain and Secure Sockets Layer (SSL) certificates were configured for secure access.

10 Testing

The testing phase was crucial for validating the application's reliability, security, and overall quality. A comprehensive testing strategy was implemented to evaluate all critical aspects of the system, ensuring it met both functional and non-functional requirements. This included unit testing to validate individual components, especially within the Django backend, and integration testing to verify seamless interaction between microservices, modules, and components, covering the front-end, backend, database, and AI opponent. System testing simulated real-world usage scenarios, including multiplayer interactions, to confirm that the system met specified requirements. End-user testing was conducted to gather feedback and ensure the system met expectations for functionality, usability, and performance.

10.1 Testing Approach

The testing approach utilized various methods to validate the system's functionality and performance. Test cases were focused on key areas such as user management (registration, login with JWT and 2FA, logout, profile updates, and password recovery), game mechanics (game rules, state management, AI behavior, and multi-player synchronization), security (authentication mechanisms and vulnerability testing), data handling (storage and integrity), and microservices communication.

10.2 Testing Phase

To improve efficiency, automation testing was integrated into the workflow. Automated unit tests were implemented for rapid feedback on individual components in the Django backend, while end-to-end tests simulated user scenarios like login, gameplay, and profile updates to ensure a seamless experience. Performance testing was performed to ensure the system could handle high loads and remain responsive, including load testing for multi-player sessions, scalability testing to support growth, and response time measurements for critical interactions like login and game updates.

To validate fixes and prevent regression, we performed extensive verification testing as follows:

- A formal process was established to log bugs, prioritize their severity, assign them to the appropriate team members, and monitor their progress until resolution. We employed a bug tracking system to ensure all issues were addressed.
- To verify that bug fixes didn't introduce new issues, we performed comprehensive regression testing by running relevant test cases and scenarios.
- We defined metrics to assess test coverage, ensuring that all critical areas of the codebase and microservices were thoroughly examined and validated.

10.3 Documentation Procedures

We utilized GitHub to carefully document and review the results of our testing, providing insights into the system's stability and performance. After each testing phase, we recorded the outcomes, tracking whether tests passed or failed and any issues that emerged. We also monitored key metrics, such as the average time to resolve issues, which helped us assess the effectiveness of our testing process and pinpoint areas for improvement.

11 Evolution

11.1 Our Project Evolution

The evolution phase of Ft_Transcendence focused on continuous improvement, addressing testing feedback and planning future enhancements. Building upon our robust testing, we refined the application's functionality, performance, and user experience.

11.2 Refinement Based on Testing and Optimization

Testing provided valuable insights for improvement. We prioritized bug fixes, enhanced multiplayer stability, optimized AI behavior, and improved UI responsiveness. Performance testing drove optimization strategies, including database query efficiency, microservice communication improvements, and real-time update enhancements through load balancing, caching, and code refactoring.

11.3 Feature Development and Enhancements

Based on user feedback, we planned feature refinements and expansions, such as UI enhancements, AI algorithm improvements, language support, and social feature integration. To streamline deployment, we automated more testing and refined our deployment procedures, including exploring further microservice deployment automation.

11.4 Documentation and Future Plans

We enhanced project documentation with detailed system architecture, microservice design, and testing procedures, and created user guides. GitHub facilitated documentation accessibility. Looking forward, we identified future developments: advanced machine learning for the AI, new game modes, granular permissions, and cloud deployment exploration. The evolution phase refined Ft_Transcendence into a robust, user-friendly application, setting the stage for future growth.

12 Conclusion

Developing this project involved implementing modern web technologies to create a seamless and engaging gaming and user experience. The focus was on integrating key aspects such as user authentication, real-time gameplay, AI-driven opponents, and security best practices while ensuring a scalable and maintainable architecture. By leveraging a structured backend, an interactive frontend, and a relational database, the application delivers smooth user interactions and reliable performance. Key features include multiplayer functionality, AI opponents, secure authentication methods, and detailed game statistics. Security was a top priority, with the implementation of Two-Factor Authentication (2FA) and JSON Web Tokens (JWT). Additionally, a microservices-based backend architecture was adopted to enhance scalability and modularity.

This report outlines the development process, covering key aspects such as project planning, system design, implementation, and testing. It is structured for clarity, with dedicated sections for risk assessment, requirement analysis, and ongoing improvements, ensuring easy navigation. Ultimately, this project showcases technical proficiency in full-stack development while emphasizing security, performance, and scalability. It reflects a well-rounded approach to modern software engineering, demonstrating the ability to build a robust and efficient web application from the ground up.

12.1 Areas for Improvement

While the application is fully functional, there are several areas for potential enhancement. A key improvement would be to increase the accessibility of the website to cater to a wider audience, including those with disabilities. Implementing features such as keyboard navigation, screen reader compatibility, and customizable color schemes could significantly improve the user experience.

Another area for improvement lies in enhancing the multiplayer experience. Currently, multiplayer and tournament modes are limited by the need for players to share the same device. A potential solution is to implement cross-device multiplayer support, allowing players to compete with others remotely, either on a 1-6 player mode or the 8-player tournament mode. This could be achieved by enabling players to log in and join a game session from their own devices, thus improving accessibility and enjoyment. Additionally, the user interface could be more customizable to further enhance player engagement. Allowing users to choose themes, adjust game settings, and even create custom avatars or profiles would not only provide a more personalized experience but also increase retention by keeping players engaged.

12.2 Addressing Limitations

Despite its many strengths, the application does have limitations. One of the major challenges is the current lack of in-game communication options, such as voice chat or text chat, which could make the multiplayer experience more interactive and dynamic. Adding these features would enhance player interaction, though it would require addressing potential issues with moderation and ensuring a secure environment. Another limitation is the scalability of the AI-driven opponents. Although AI currently provides a reasonable challenge, it is not highly adaptable to different playstyles. A more sophisticated AI system could be developed to better mimic human players, offering a more challenging experience for users at all levels. However, this would require additional resources and could affect performance.

Moreover, as the player base grows, maintaining consistent performance across all users could become a challenge. While the microservices-based architecture provides scalability, it may require additional optimization to handle higher volumes of simultaneous players without compromising game quality. Lastly, there is a need for more comprehensive analytics and reporting tools, both for players and administrators. Features like tracking player performance over time, suggesting personalized gameplay strategies, or even organizing events and leagues could enrich the overall experience but would require additional development effort.

12.3 Final Thoughts

In conclusion, while the project demonstrates a high level of technical competency and effectively delivers a smooth and secure gaming experience, there are several areas for potential enhancement. Future improvements could focus on expanding the accessibility and interactivity of the platform, increasing customization options, and addressing scalability and performance challenges to ensure long-term sustainability and a continuously engaging user experience.

13 References

- [1] U. of Wisconsin-Madison, “Writing an abstract for a research paper.” [Online]. Available: <https://writing.wisc.edu/handbook/assignments/writing-an-abstract-for-your-research-paper/>
- [2] TechTarget, “Waterfall vs. agile vs. iterative development.” [Online]. Available: <https://www.techtarget.com/searchsoftwarequality/tip/Waterfall-vs-Agile-vs-iterative-development-explained>
- [3] GitHub, “Github docs.” [Online]. Available: <https://docs.github.com>
- [4] Overleaf, “Overleaf: Latex documentation and writing.” [Online]. Available: <https://www.overleaf.com/>
- [5] I. D. Foundation, “Wireframes and ui design.” [Online]. Available: <https://www.interaction-design.org/literature/topics/wireframe>
- [6] NoteGPT, “Ai flowchart generator.” [Online]. Available: <https://notegpt.io/ai-flowchart-generator>
- [7] Django Software Foundation, “Django official documentation.” [Online]. Available: <https://docs.djangoproject.com>

14 Appendices

14.1 1. Homepage



Figure 8: Homepage



Figure 9: Instruction Page

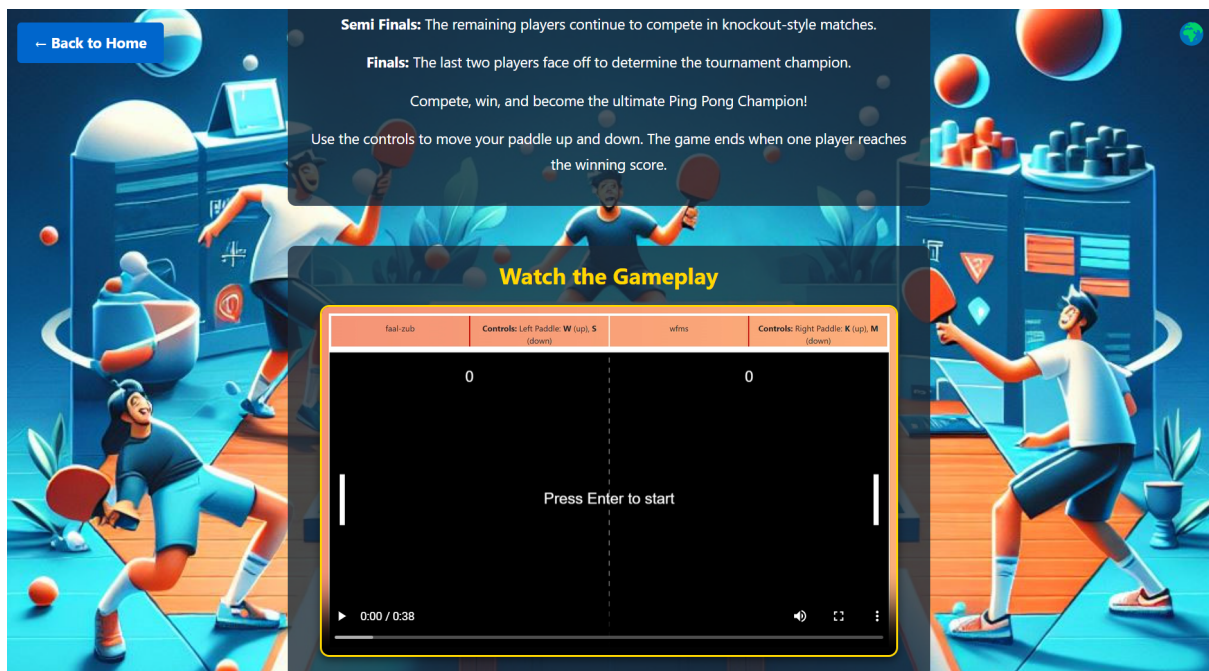


Figure 10: Instruction Page - Video

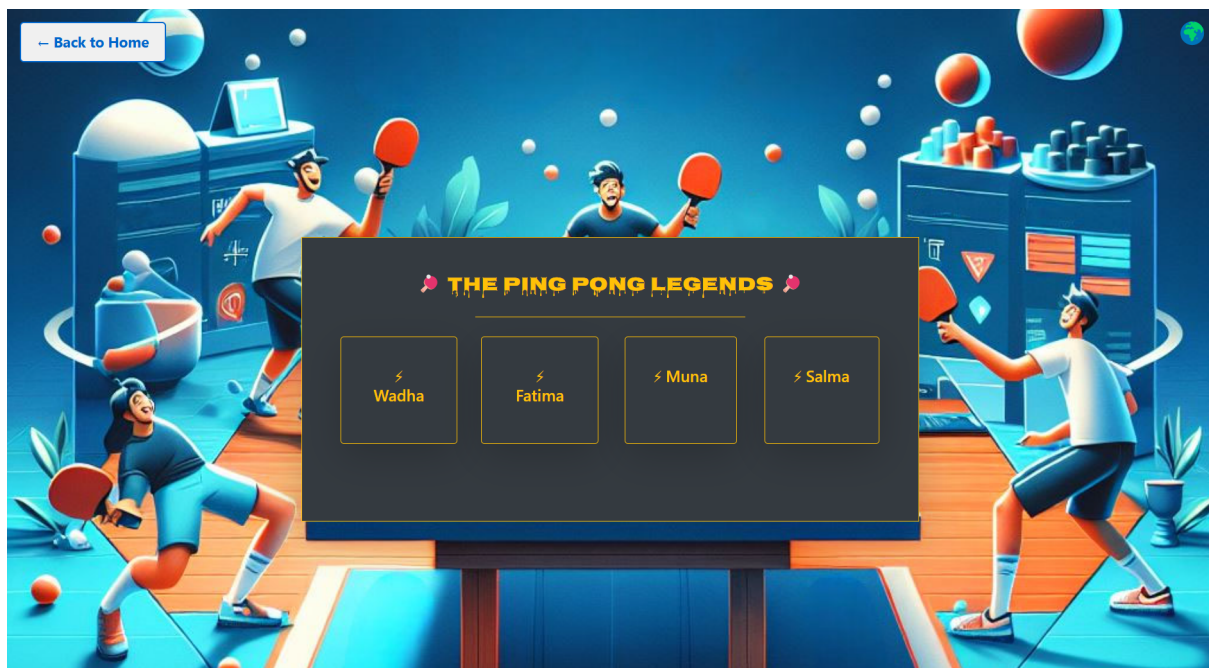
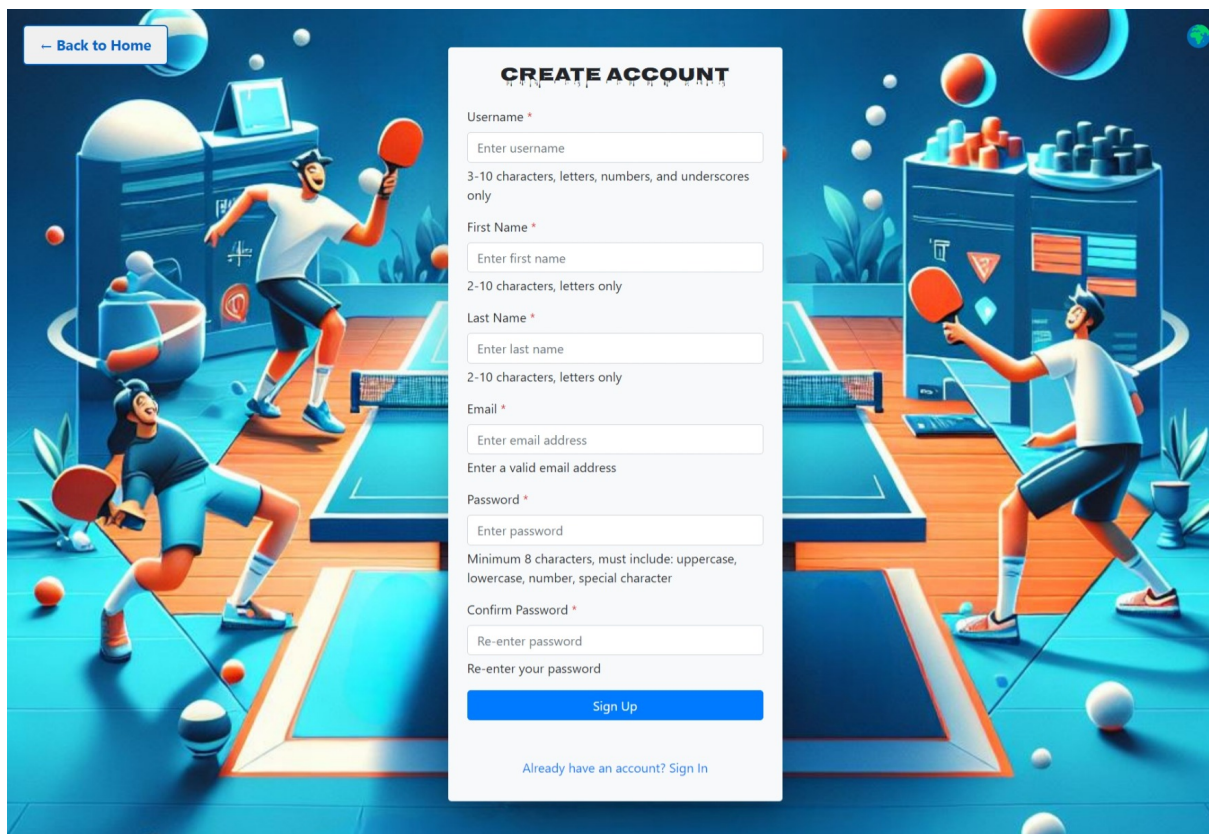


Figure 11: About Us Page

14.2 2. Sign Up



← Back to Home

CREATE ACCOUNT

Username *

3-10 characters, letters, numbers, and underscores only

First Name *

2-10 characters, letters only

Last Name *

2-10 characters, letters only

Email *

Enter a valid email address

Password *

Minimum 8 characters, must include: uppercase, lowercase, number, special character

Confirm Password *

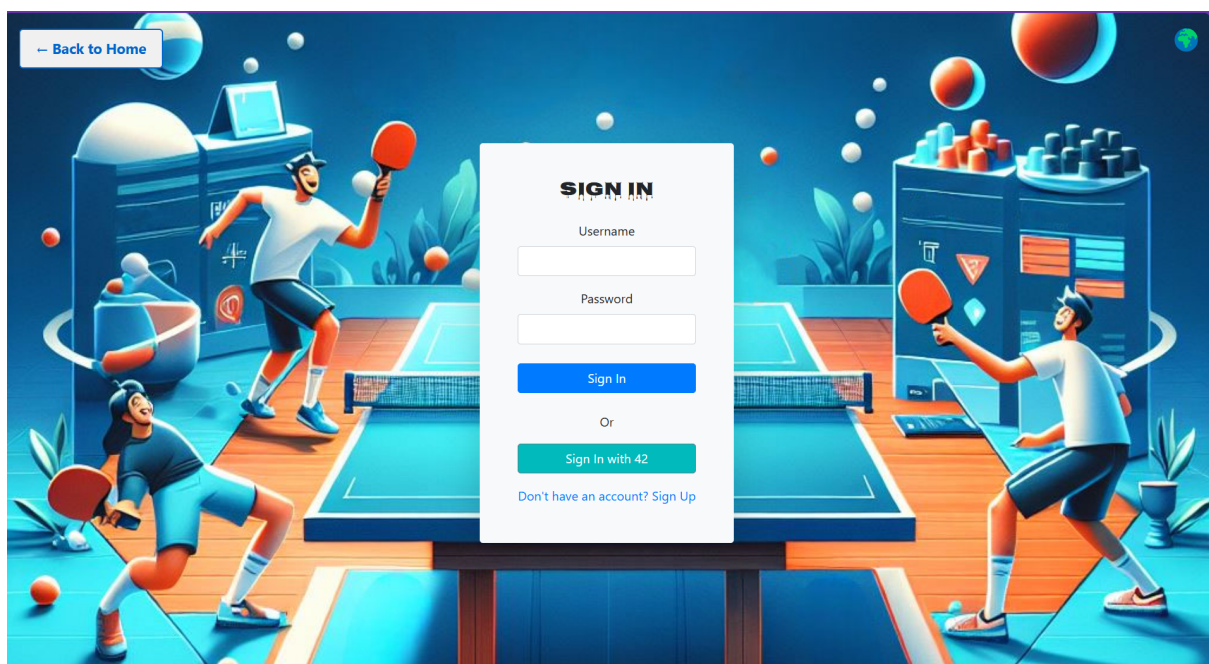
Re-enter your password

[Sign Up](#)

[Already have an account? Sign In](#)

Figure 12: Sign Up Page

14.3 3. Sign In



← Back to Home

SIGN IN

Username

Password

[Sign In](#)

Or

[Sign In with 42](#)

[Don't have an account? Sign Up](#)

Figure 13: Sign In Page

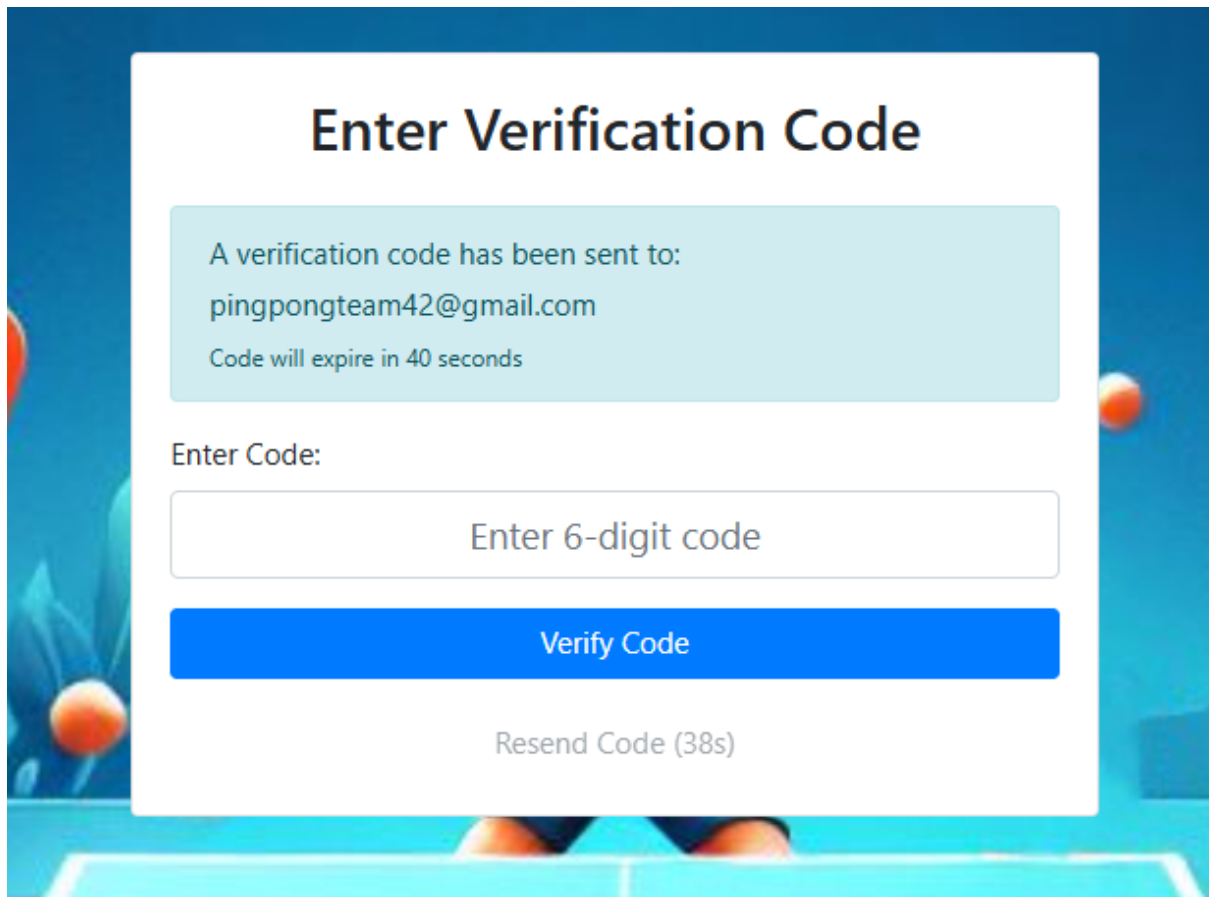


Figure 14: Verify Code (2FA) Page

14.4 4. Game Hub

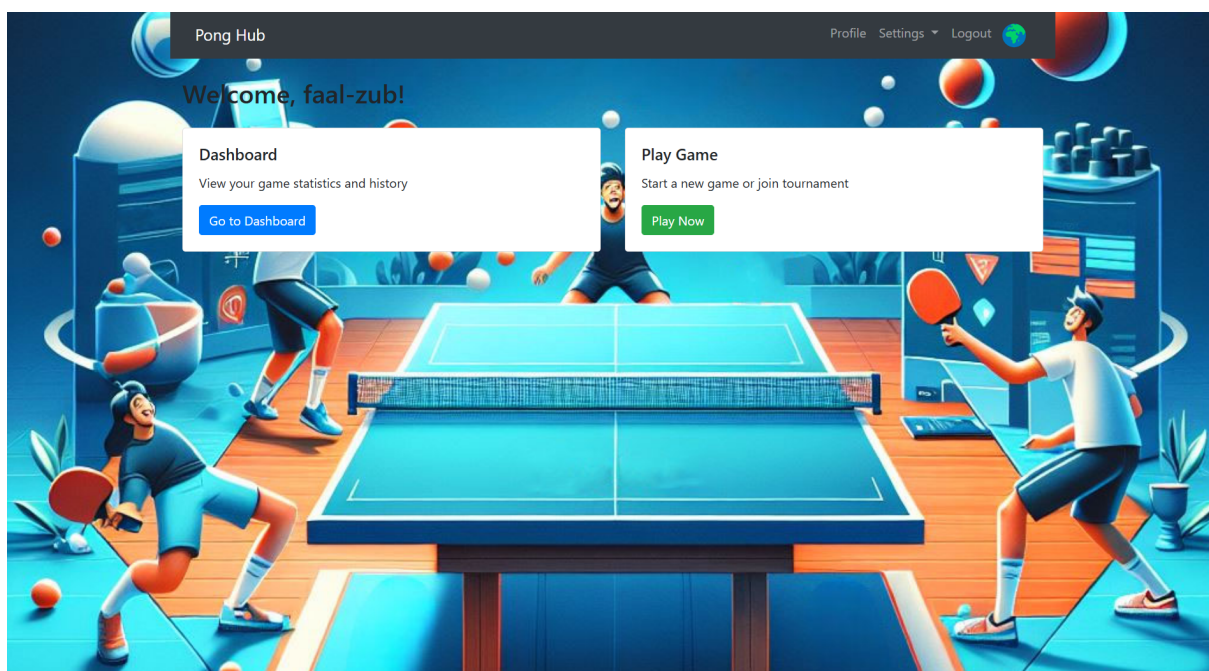


Figure 15: Game Hub Page

14.5 5. Dashboard

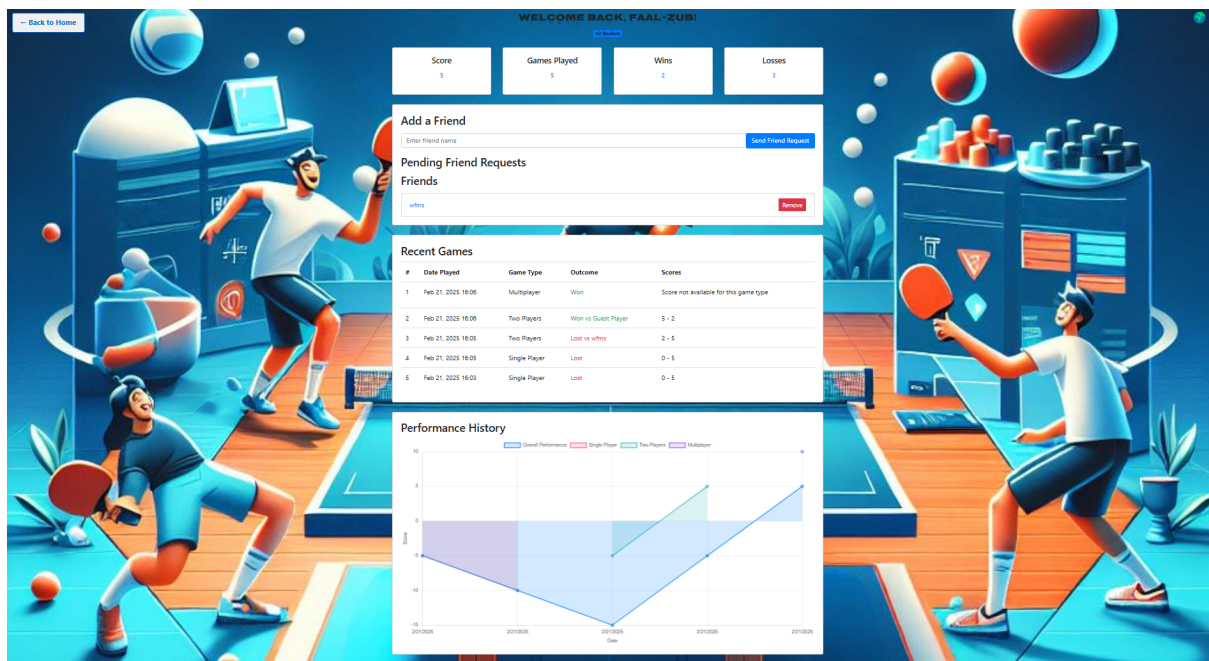


Figure 16: Dashboard - User Stats and Match History

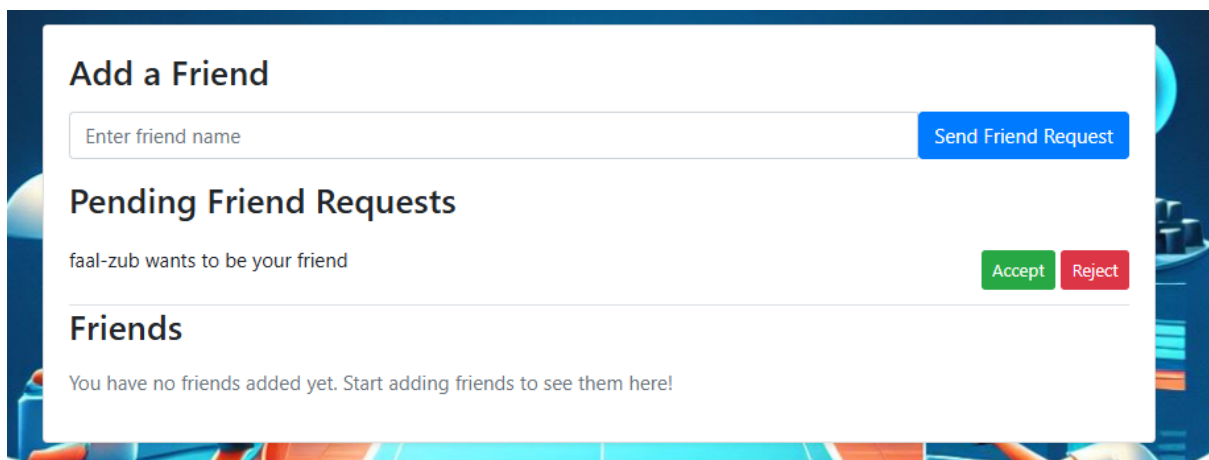


Figure 17: Friend Requests - Accept/Reject

14.6 6. Profile

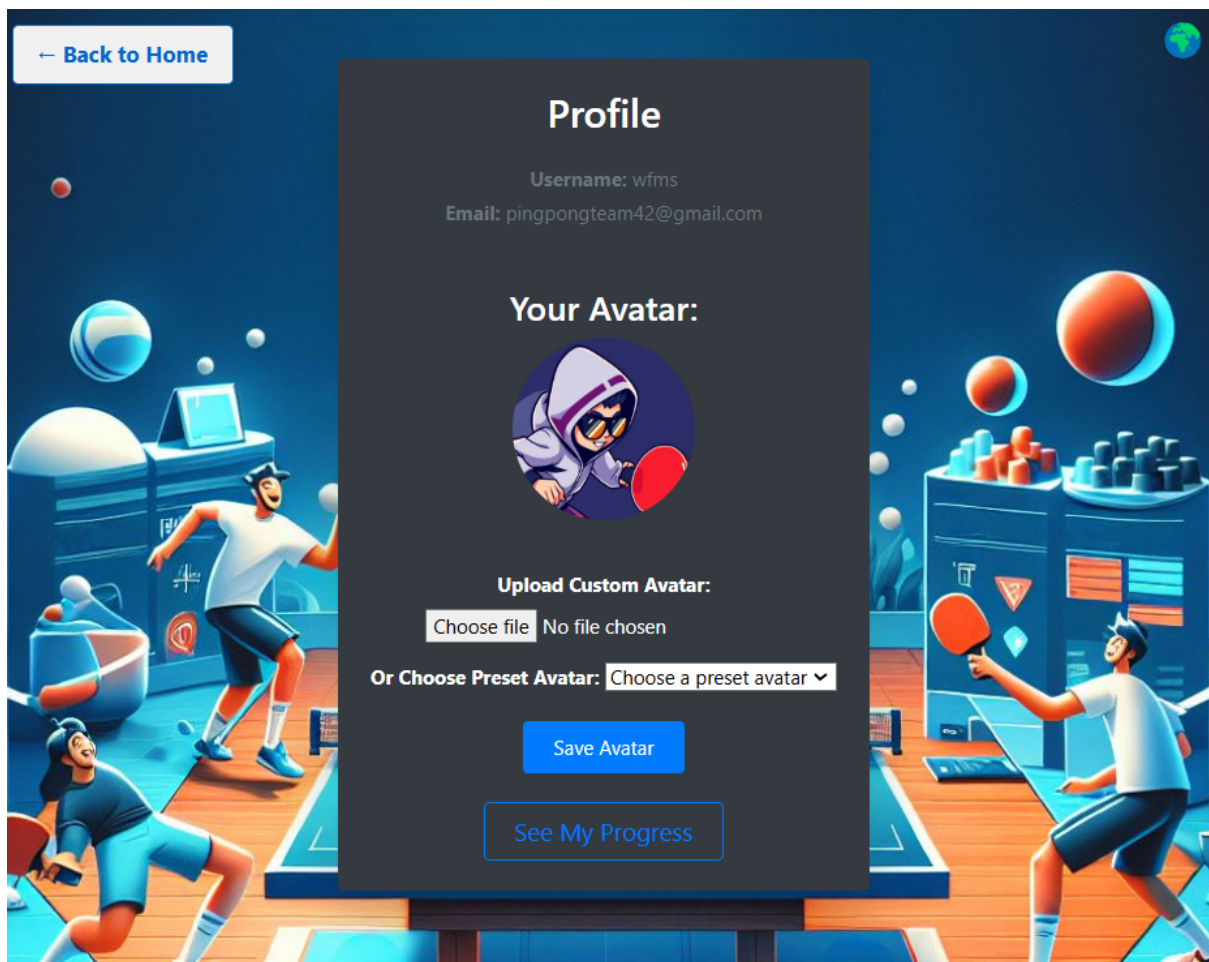


Figure 18: User Profile

14.7 7. Menu (Game Modes)

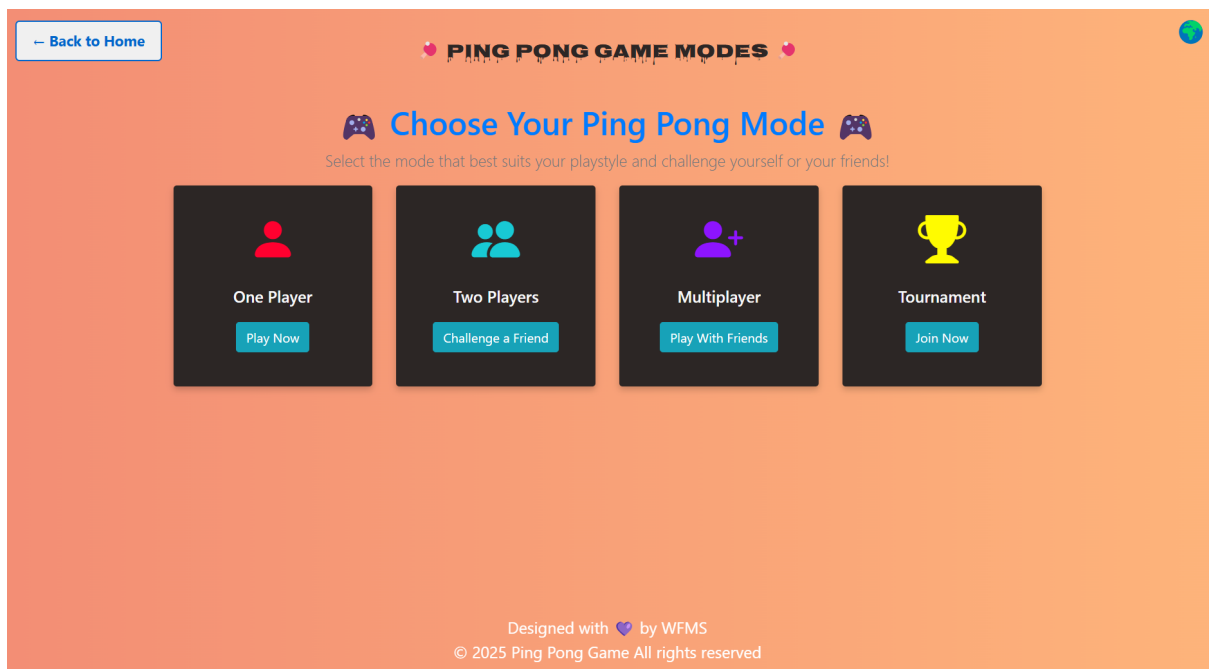


Figure 19: Game Menu - Select Game Mode

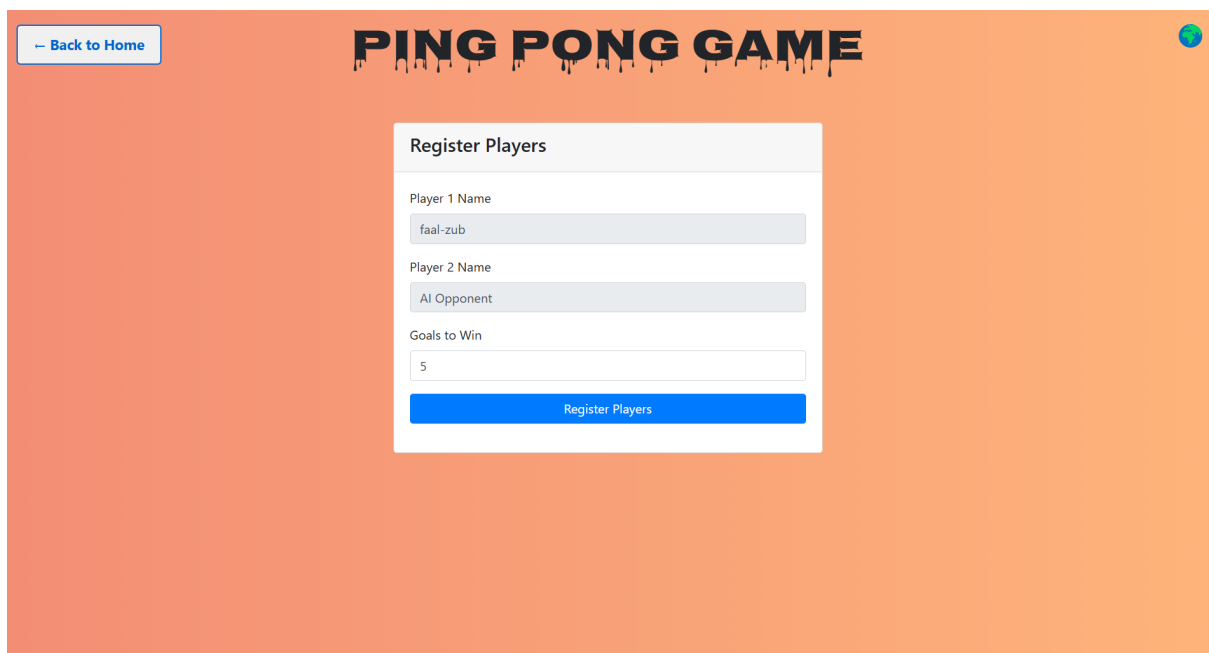


Figure 20: Single Player Setup

[← Back to Home](#)

PING PONG GAME

Register Players

Player 1 Name
faal-zub

Player 2 Name
Enter player name

Goals to Win
5

Register Players

Figure 21: Two Player Setup

[← Back to Home](#)

WELCOME TO MULTI-PONG

Major Player:
faal-zub

Enter the number of players
(1-6):
6

Start Game

Figure 22: Multiplayer Setup

[← Back to Home](#)



PING PONG TOURNAMENT

Register Players (Enter 8 player names)

Player Name 1

Player Name 2

Player Name 3

Player Name 4

Player Name 5

Player Name 6

Player Name 7

Player Name 8

Register Players

Figure 23: Tournament Setup

14.8 8. Settings

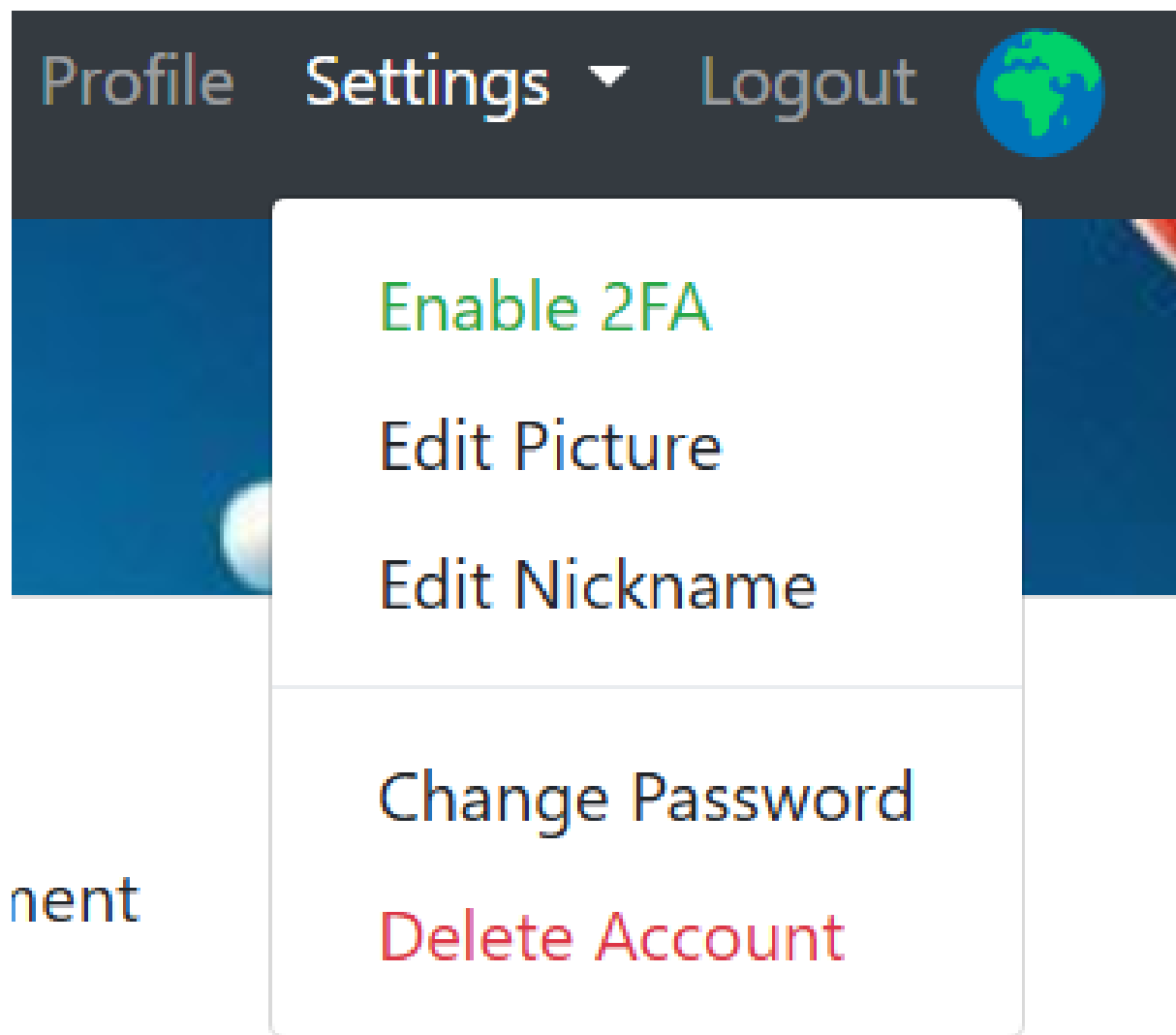


Figure 24: Settings Page

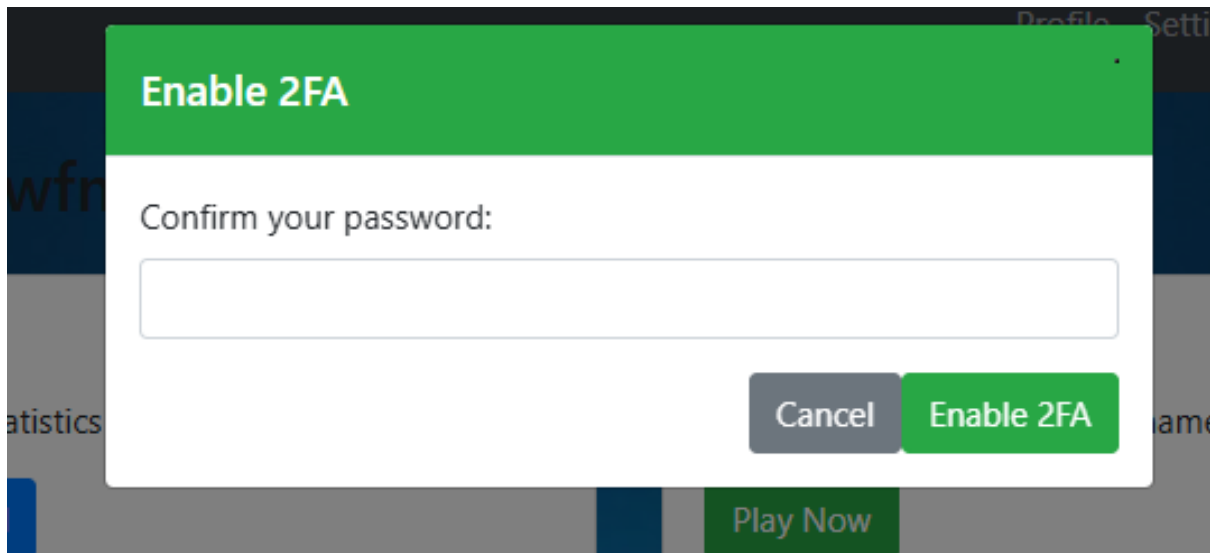


Figure 25: Enable 2FA

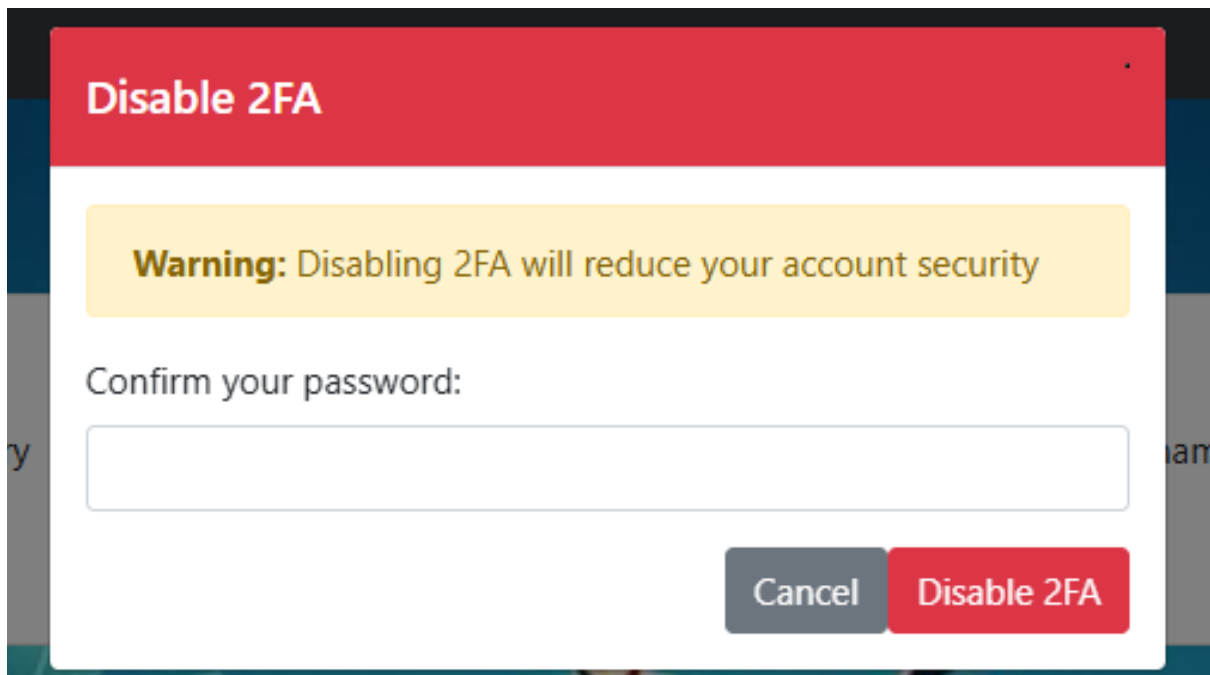
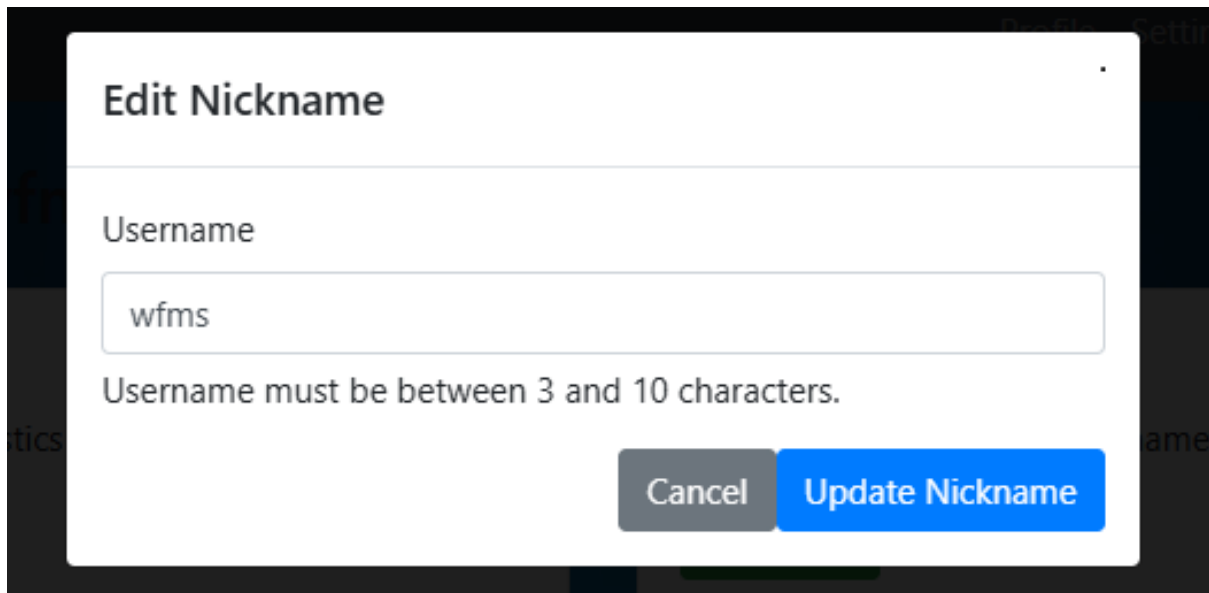


Figure 26: Disable 2FA



The image shows a modal dialog box titled "Edit Nickname". It contains a label "Username" above a text input field that contains the text "wfms". Below the input field is a message: "Username must be between 3 and 10 characters." At the bottom right of the dialog are two buttons: a grey "Cancel" button and a blue "Update Nickname" button.

Edit Nickname

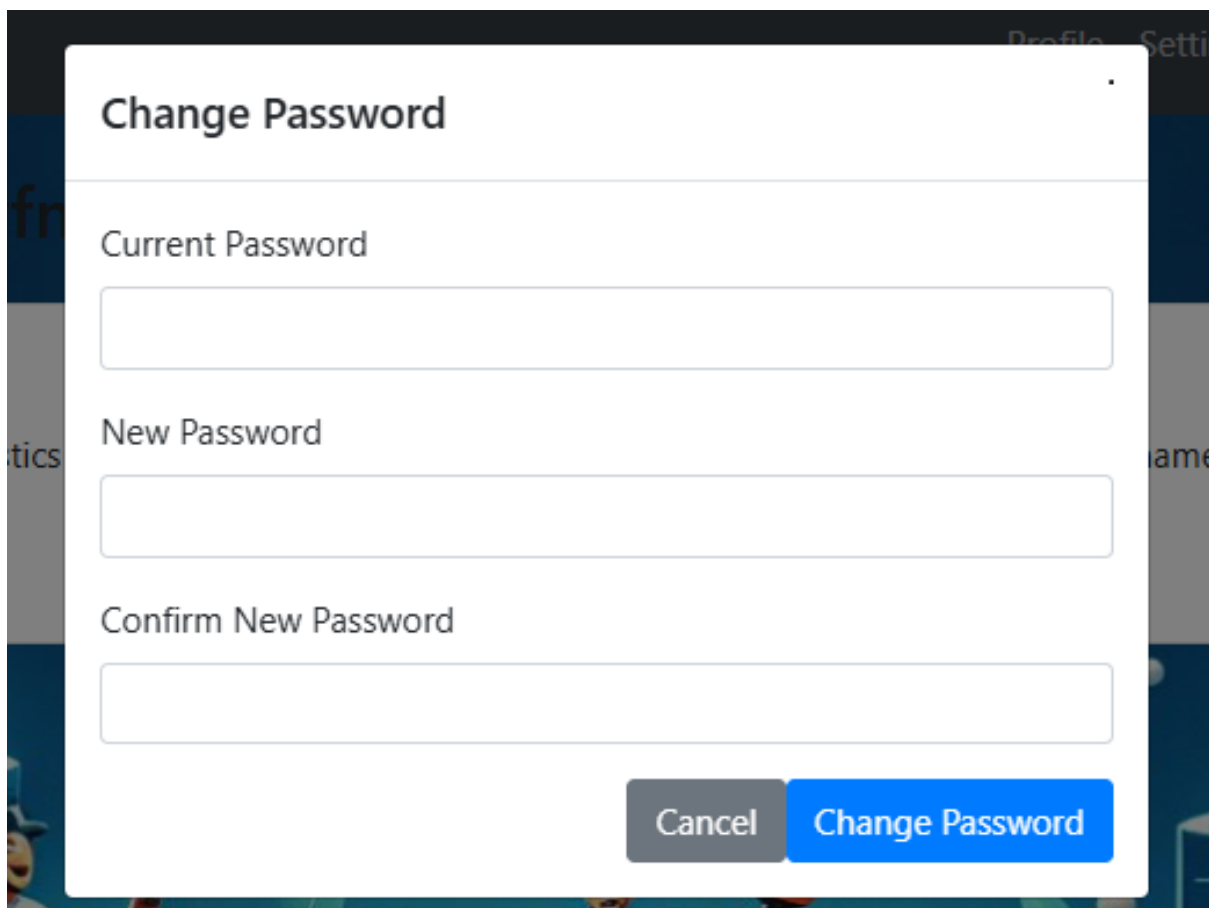
Username

wfms

Username must be between 3 and 10 characters.

Cancel Update Nickname

Figure 27: Edit Nickname



The image shows a modal dialog box titled "Change Password". It contains three labels with corresponding text input fields: "Current Password", "New Password", and "Confirm New Password". At the bottom right of the dialog are two buttons: a grey "Cancel" button and a blue "Change Password" button.

Change Password

Current Password

New Password

Confirm New Password

Cancel Change Password

Figure 28: Edit Password

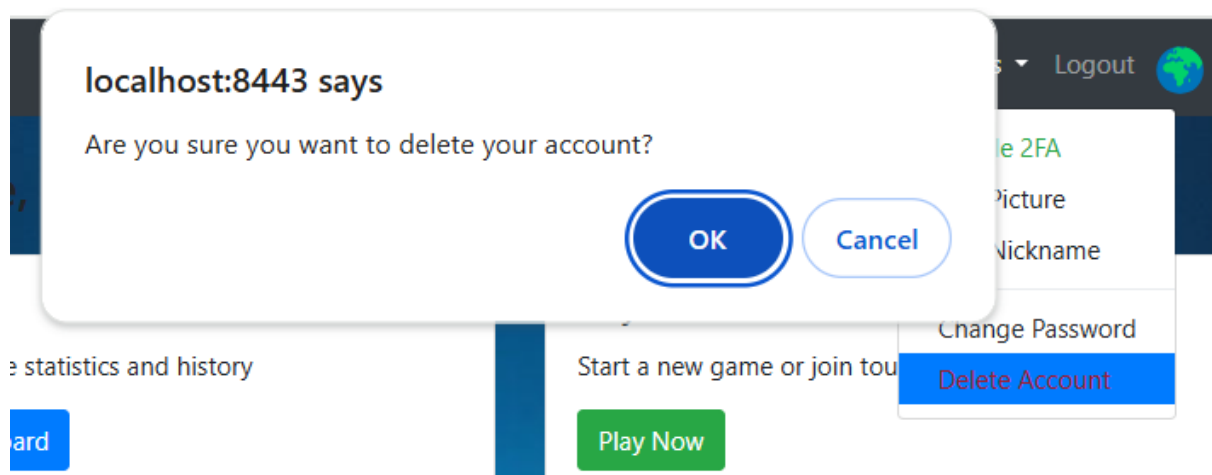


Figure 29: Delete Account

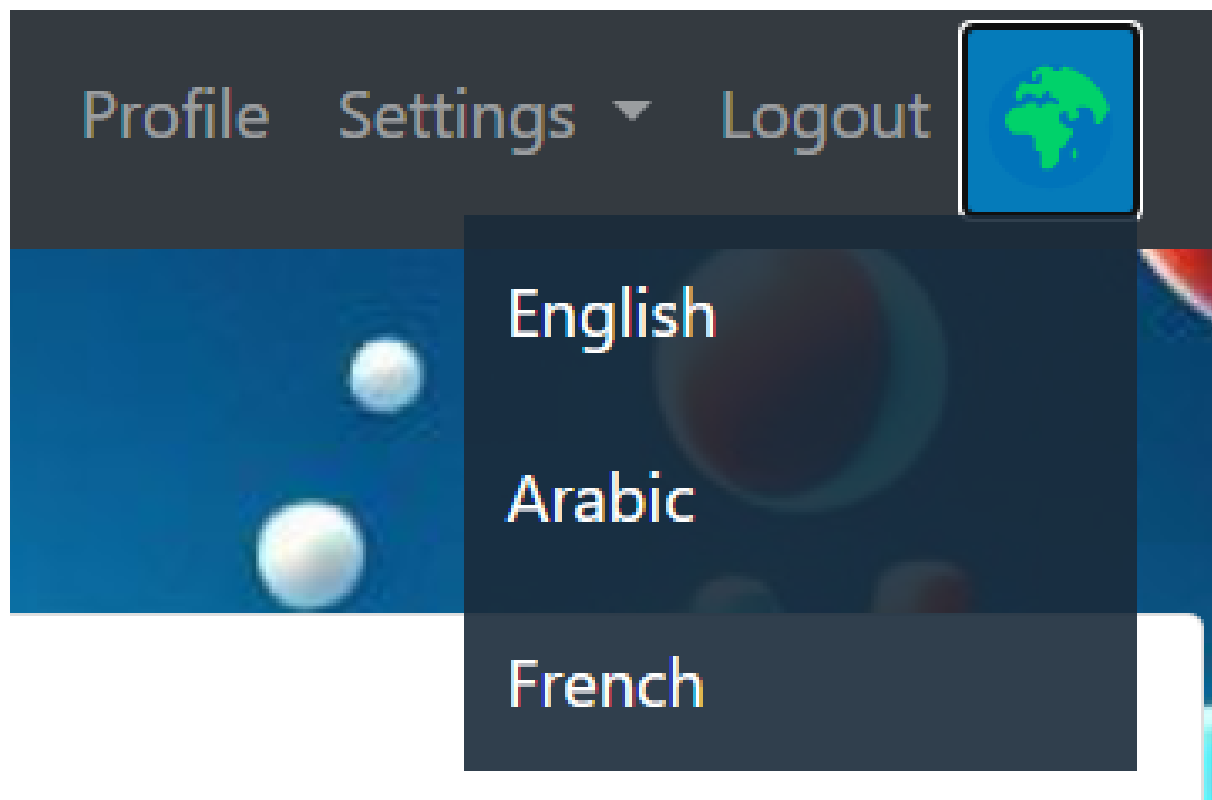


Figure 30: Language Settings